AromaSDK

Lightweight development toolkit for cross-platform applications on Linux, Android, Windows, and embedded systems.

Version 0.0.1 (alpha)
Generated June 02, 2026
© 2026

Contents

Getting Started

This page provides a comprehensive guide for new developers to set up the AromaUI environment, create a new project, and deploy a "Hello World" application across Linux, Android, and Web platforms.

1. Environment Setup

The AromaUI toolchain is centered around a Python-based CLI utility named `aroma`. This tool abstracts complex build systems like CMake, Gradle, and Emscripten to provide a unified developer experience.

1.1. Cloning the Repository

To begin, clone the repository and its submodules. The `--recursive` flag is mandatory as AromaUI relies on several vendor dependencies (e.g., FreeType, GLES headers) located in submodules docs/getting-started/installation.md3-9

```
git clone https://github.com/BinaryInkTN/AromaUI.git --recursive
```

1.2. Installing the CLI

The `aroma` command is a wrapper script located at `bin/aroma` bin/aroma1-10 which invokes the core logic in `tools/cli/aroma.py` tools/cli/aroma.py1-138 To use it globally:

1. Locate the `bin/` directory within the cloned repository.
2. Add this directory to your system's `PATH` environment variable docs/getting-started/installation.md11-12

1.3. Dependency Validation (`aroma doctor`)

The `aroma doctor` command validates your local environment, checking for essential tools such as CMake, GCC, Java, and the Android SDK/NDK docs/getting-started/installation.md18-31

Toolchain Validation Logic The `aroma doctor` command executes the `AndroidSDK` class methods to locate existing installations in common paths like `~/Android/Sdk` or `/usr/lib/android-sdk` tools/cli/aroma.py144-164

Sources: `bin/aroma`, `tools/cli/aroma.py`, `docs/getting-started/installation.md`

---

2. Toolchain Management

AromaUI provides automated installation for mobile development dependencies to ensure version compatibility with the framework's internal requirements (e.g., NDK 25.1.8937393) tools/cli/aroma.py19

2.1. Android SDK Installation

If `aroma doctor` reports missing Android components, run:

```
aroma install-sdk
```

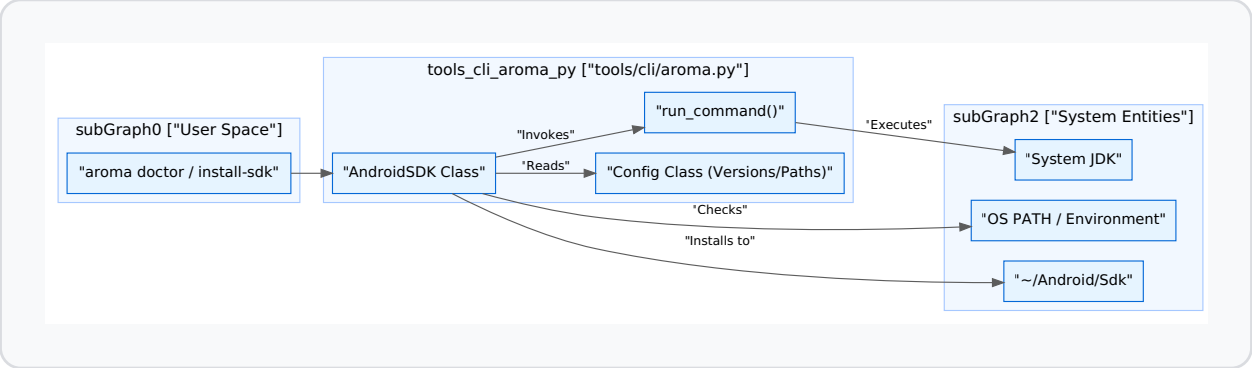
This triggers the `AndroidSDK.install()` method, which performs the following:

1. **Downloads Command Line Tools**: Fetches the platform-specific zip from Google's repositories tools/cli/aroma.py183-215
2. **Accepts Licenses**: Automatically handles the `sdkmanager --licenses` handshake tools/cli/aroma.py175
3. **Installs Packages**: Downloads specific versions of `platform-tools`, `platforms;android-34`, `build-tools`, `ndk`, and `cmake` as defined in the `Config` class tools/cli/aroma.py18-29

2.2. CLI Data Flow: Setup to Execution

The following diagram illustrates how the `aroma` CLI interacts with the system to prepare the development environment.

CLI Environment Initialization



Sources: `tools/cli/aroma.py`, `docs/getting-started/installation.md`

---

3. Creating a Project (`aroma create`)

The `aroma create` command initializes a new project using the `ProjectCreator` engine. It scaffolds a directory structure compatible with all supported backends.

3.1. Scaffolding Process

When you run `aroma create`, the CLI prompts for a project name and package name (e.g., `com.example.hello`) tools/cli/aroma.py124-125 It then performs placeholder substitution using templates:

- **Android**: Generates `AndroidManifest.xml`, `build.gradle`, and the `AromaHelper.java` JNI bridge tools/cli/templates/android/app/build.gradle1-34
- **CMake**: Configures `CMakeLists.txt` to link against the AromaUI core and appropriate backends.

3.2. Project Structure

Path	Description
`src/main.c`	Entry point for the application.
`src/main/cpp/`	Native C code and CMake configuration for Android.
`src/main/java/`	Java source for Android lifecycle and JNI helpers.
`CMakeLists.txt`	Root build configuration for Linux and Web.

Sources: `tools/cli/aroma.py`, `tools/cli/templates/android/app/build.gradle`

---

4. Running the Hello World Example

A minimal AromaUI application follows a strict lifecycle: initialization, window creation, scene graph construction, and the main execution loop.

4.1. Implementation Detail (`main.c`)

The application state is typically managed in a central `AppState` struct docs/getting-started/hello-world.md22-33

1. **Initialization**: `aroma_ui_init()` must be the first call to set up internal slab allocators and global state docs/getting-started/hello-world.md51-57
2. **Windowing**: `aroma_ui_create_window()` initializes the platform backend (e.g., GLPS for Linux or ANativeWindow for Android) docs/getting-started/hello-world.md76-81
3. **UI Construction**: Widgets are created using factory functions like `aroma_ui_container()` and `aroma_ui_button()`, which append nodes to the `AromaNode` tree docs/getting-started/hello-world.md119-172

4. **Main Loop**: The application enters a loop calling `aroma_ui_process_events()` to handle input and `aroma_ui_render()` to trigger the layout/draw pipeline docs/getting-started/hello-world.md204-210

4.2. Platform Deployment Commands

Once the code is written, use the CLI to run it on various targets:

- **Linux**:

```
aroma run linux
```

Compiles using the local GCC/Clang and runs the binary using the GLPS (OpenGL Linux Platform Services) backend. - **Android**:

```
aroma run android --emu
```

Compiles the native code via NDK, builds the APK via Gradle, and deploys it to a running Android Virtual Device (AVD). It uses the `android_main` entry point to bridge with `android_native_app_glue` docs/getting-started/hello-world.md249-261 - **Web (WASM)**:

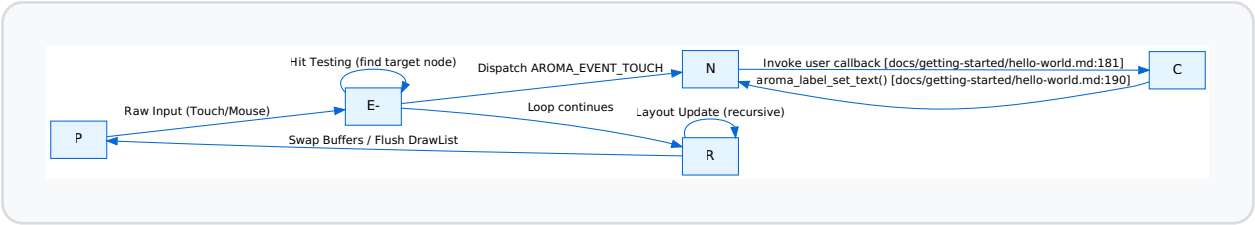
```
aroma run web
```

Uses Emscripten to compile the C code into WebAssembly, targeting the HTML5 Canvas via the Emscripten platform backend.

4.3. Data Flow: From Event to Screen

The following diagram bridges the high-level application logic to the internal code entities that process a button click in the Hello World example.

Event Handling and Rendering Flow



Sources: `docs/getting-started/hello-world.md`, `docs/getting-started/architecture.md`, `docs/ui/layouts.md`

Architecture Overview

AromaUI is built on a strict four-layer architecture designed to provide native performance while maintaining high portability across desktop (Linux), mobile (Android), and resource-constrained embedded systems (ESP32/TFT). The framework enforces a clear separation between application logic, UI state management, platform services, and hardware-accelerated rendering.

System Hierarchy

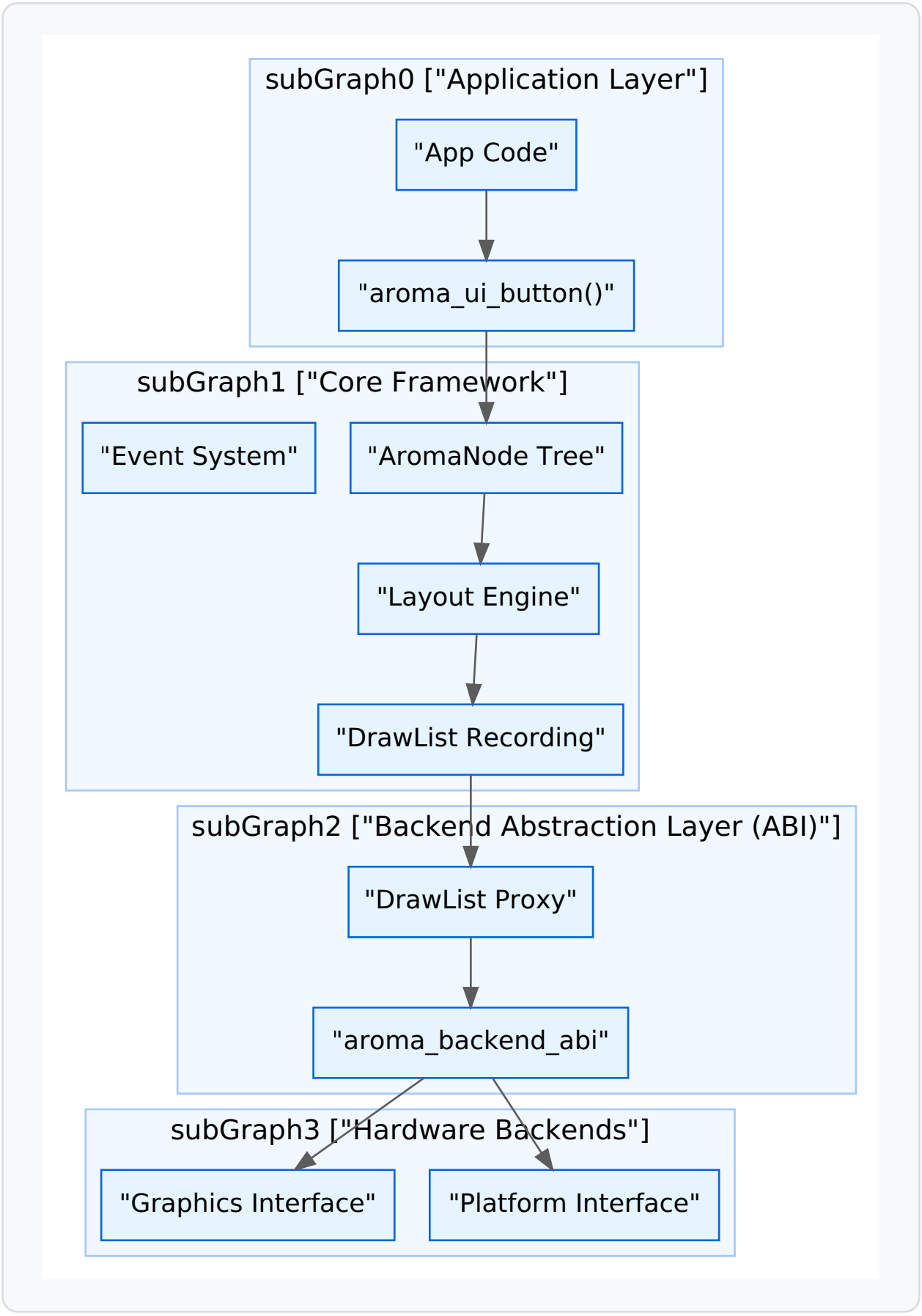
The system is organized into the following layers:

1. **Application Layer**: Consumer code using the `aroma_ui_*` factory functions and high-level widget APIs include/aroma_ui.h135-158
2. **Core Framework**: Manages the `AromaNode` scene graph, the recursive layout engine, the event dispatch system, and the deferred rendering pipeline src/core/aroma_ui_impl.c165-166
3. **Backend Abstraction Layer (ABI)**: A proxy layer (`AromaBackendABI`) that routes generic draw and platform commands to the appropriate backend src/backends/aroma_abi.c27-35
4. **Platform & Graphics Backends**: Hardware-specific implementations for input, windowing (GLPS, GLFW, Android), and rendering (GLES3, Vulkan, TFT_eSPI) src/backends/aroma_abi.h6-20

Architectural Flow

The following diagram illustrates the data flow from a high-level widget creation to low-level hardware execution.

Diagram: System Layer Interaction


subGraph0 ["Application Layer"]
"App Code"
"aroma_ui_button()"
subGraph1 ["Core Framework"]
"Event System"
"AromaNode Tree"
"Layout Engine"
"DrawList Recording"
subGraph2 ["Backend Abstraction Layer (ABI)"]
"DrawList Proxy"
"aroma_backend_abi"
subGraph3 ["Hardware Backends"]
"Graphics Interface"
"Platform Interface"

Sources:src/core/aroma_ui_impl.c1-15src/backends/aroma_abi.h27-35include/aroma_ui.h1-23

Core Concepts

AromaNode (The Scene Graph)

Every UI element in AromaUI is an `AromaNode`. It is the base unit of the scene graph, holding geometric data, visibility states, and parent-child relationships include/aroma_ui.h36-62 Nodes are allocated via a slab allocator to ensure deterministic performance on embedded targets src/core/aroma_ui_impl.c7-8

Dirty-Region Tracking

AromaUI employs an invalidation-based rendering model. When a node's state changes (e.g., a button is pressed), it is marked as "dirty" src/core/aroma_ui_impl.c221-222 The framework tracks these regions and only re-renders windows that have entries in the global dirty list src/core/aroma_ui_impl.c225-229

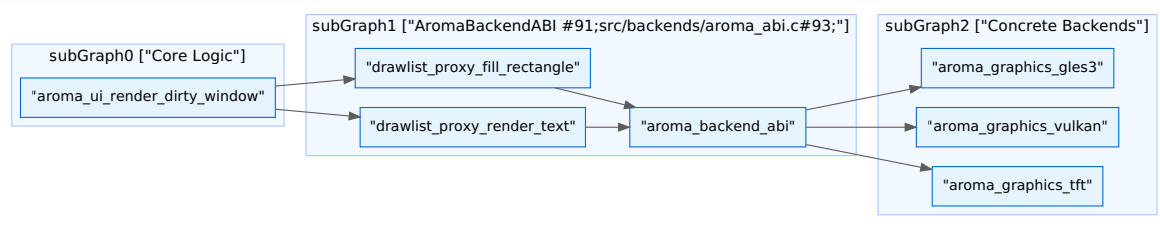
DrawList and Deferred Rendering

To optimize batching, AromaUI does not issue immediate draw calls. Instead, it records commands into an `AromaDrawList`src/core/aroma_ui_impl.c62 During the render phase, these commands are sorted (e.g., by Z-index) and then flushed to the graphics backend through the ABI src/core/aroma_ui_impl.c71-75

Backend Abstraction Layer (ABI)

The `AromaBackendABI` acts as a central switchboard. It allows the Core Framework to remain platform-agnostic while supporting diverse environments like Android or surfaceless EGL clusters src/backends/aroma_abi.c13-32

Diagram: ABI Proxy Pattern



Sources:src/backends/aroma_abi.c51-63src/backends/aroma_abi.c93-104src/backends/aroma_abi.h27-35

Data Flow: Frame Lifecycle

The frame lifecycle is managed by the platform's event loop (e.g., GLFW or Android's `Choreographer`) src/backends/platforms/aroma_platform_interface.h99-103

Phase	Description	Key Functions
Input	Platform captures raw input and sends to Event System.	`run_event_loop` src/backends/platforms/aroma_platform_interface.h103
Layout	Recursive pass to calculate node bounds.	`aroma_node_update_layout` src/widgets/aroma_window.c38
Record	Nodes record draw commands into the `AromaDrawList`.	`aroma_drawlist_cmd_fill_rect` src/backends/aroma_abi.c56
Batch	Sort tasks by Z-index and apply frustum culling.	`collect_draw_tasks` src/core/aroma_ui_impl.c71-73
Flush	ABI routes batched commands to the Graphics Backend.	`swap_buffers` src/backends/platforms/aroma_platform_interface.h108

Sources:src/core/aroma_ui_impl.c68-75src/backends/platforms/aroma_platform_interface.h35-108src/widgets/aroma_window.c32-42

Animation Engine

The AromaUI Animation Engine is a lightweight, retained-mode subsystem designed to provide smooth property transitions for `AromaNode` elements. It operates by interpolating values over time using various easing functions and automatically integrating with the framework's dirty-region tracking and layout phases to trigger necessary redraws.

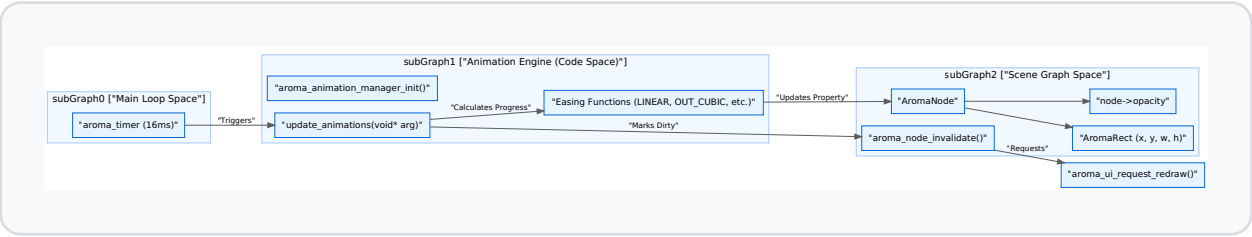
System Architecture

The animation engine is driven by a dedicated timer that updates at approximately 60 FPS (16ms intervals). It maintains a global linked list of active `AromaAnimation` structures, processing them sequentially during each tick.

Animation Data Flow

The following diagram illustrates how the animation engine interacts with the core UI loop and the scene graph.

Animation Update Lifecycle



Sources: [src/core/aroma_animation.c17-113src/core/aroma_animation.c115-119include/aroma_animation.h32-46]

Animation Types

AromaUI supports built-in transitions for common geometric and visual properties, as well as a custom mode for widget-specific logic.

Type	Enum Constant	Property Modified
Slide X	`AROMA_ANIM_SLIDE_X`	`node->rect.x`
Slide Y	`AROMA_ANIM_SLIDE_Y`	`node->rect.y`
Scale X	`AROMA_ANIM_SCALE_X`	`node->rect.width`
Scale Y	`AROMA_ANIM_SCALE_Y`	`node->rect.height`
Fade	`AROMA_ANIM_FADE`	`node->opacity`
Custom	`AROMA_ANIM_CUSTOM`	User-defined via `AromaAnimationCallback`

Sources: include/aroma_animation.h10-18src/core/aroma_animation.c81-100

Easing Functions

Easings control the rate of change for the `current_val` relative to time progress. The default easing for all animations is `AROMA_EASE_OUT_CUBIC`.

- **Linear**: `AROMA_EASE_LINEAR` - Constant speed.
- **Quadratic**: `AROMA_EASE_IN_QUAD`, `AROMA_EASE_OUT_QUAD`, `AROMA_EASE_IN_OUT_QUAD`.
- **Cubic**: `AROMA_EASE_OUT_CUBIC` - Smooth deceleration.
- **Back**: `AROMA_EASE_OUT_BACK` - Slight overshoot before settling.
- **Elastic**: `AROMA_EASE_OUT_ELASTIC` - Damped oscillation effect.

Sources: include/aroma_animation.h20-28src/core/aroma_animation.c45-78

API Reference

Initialization and Management

The engine must be initialized before use, typically during application startup.

- `aroma_animation_manager_init()`: Creates the global 16ms timer and prepares the animation list. src/core/aroma_animation.c115-119
- `aroma_animation_start(target, type, start, end, duration)`: Begins an animation. If the `target` node already has a running animation, it is stopped first. src/core/aroma_animation.c121-142
- `aroma_animation_stop(target)`: Flags all animations on the target node as `is_running = false`, allowing them to be garbage collected on the next tick. src/core/aroma_animation.c144-152

Custom Animations

For properties not handled by the standard engine (e.g., color blending or rotation), `aroma_animation_start_custom` allows passing a callback.

```
// Definition of the custom callback
typedef void (*AromaAnimationCallback)(AromaNode* target, float current_val, void* user_dat
```

Sources: include/aroma_animation.h30src/core/aroma_animation.c155-162

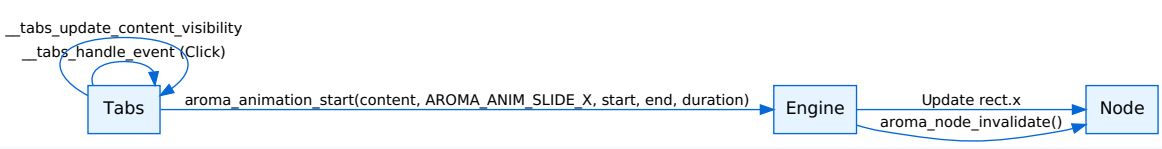
Integration with Layout and Widgets

Animations are deeply integrated into the layout phase. When an animation updates a node's property (like `rect.x`), it calls `aroma_node_invalidate(target)`. This marks the node's region as dirty, forcing the `aroma_ui_request_redraw` call at the end of the update tick. src/core/aroma_animation.c102-112

Example: Tab Transitions

The `AromaTabs` widget uses the animation engine to provide sliding transitions when switching content.

Tab Animation Logic



Sources: src/widgets/aroma_tabs.c124-133src/widgets/aroma_tabs.c158-174

Implementation Details

The Update Loop

The `update_animations` function is the core of the engine. It performs the following steps:

1. **Garbage Collection**: Removes animations where `is_running` is false. src/core/aroma_animation.c27-36
2. **Progress Calculation**: Computes `progress` as `(now - start_time) / duration`. src/core/aroma_animation.c38-42
3. **Interpolation**: Applies the selected `AromaEasingType` to transform linear progress into eased progress. src/core/aroma_animation.c45-78
4. **Property Application**: Writes the calculated `current_val` directly to the `AromaNode` or invokes the `custom_cb`. src/core/aroma_animation.c81-100
5. **Invalidation**: Triggers a UI redraw request if any animation was updated. src/core/aroma_animation.c110-112

Sources: src/core/aroma_animation.c17-113

Event System

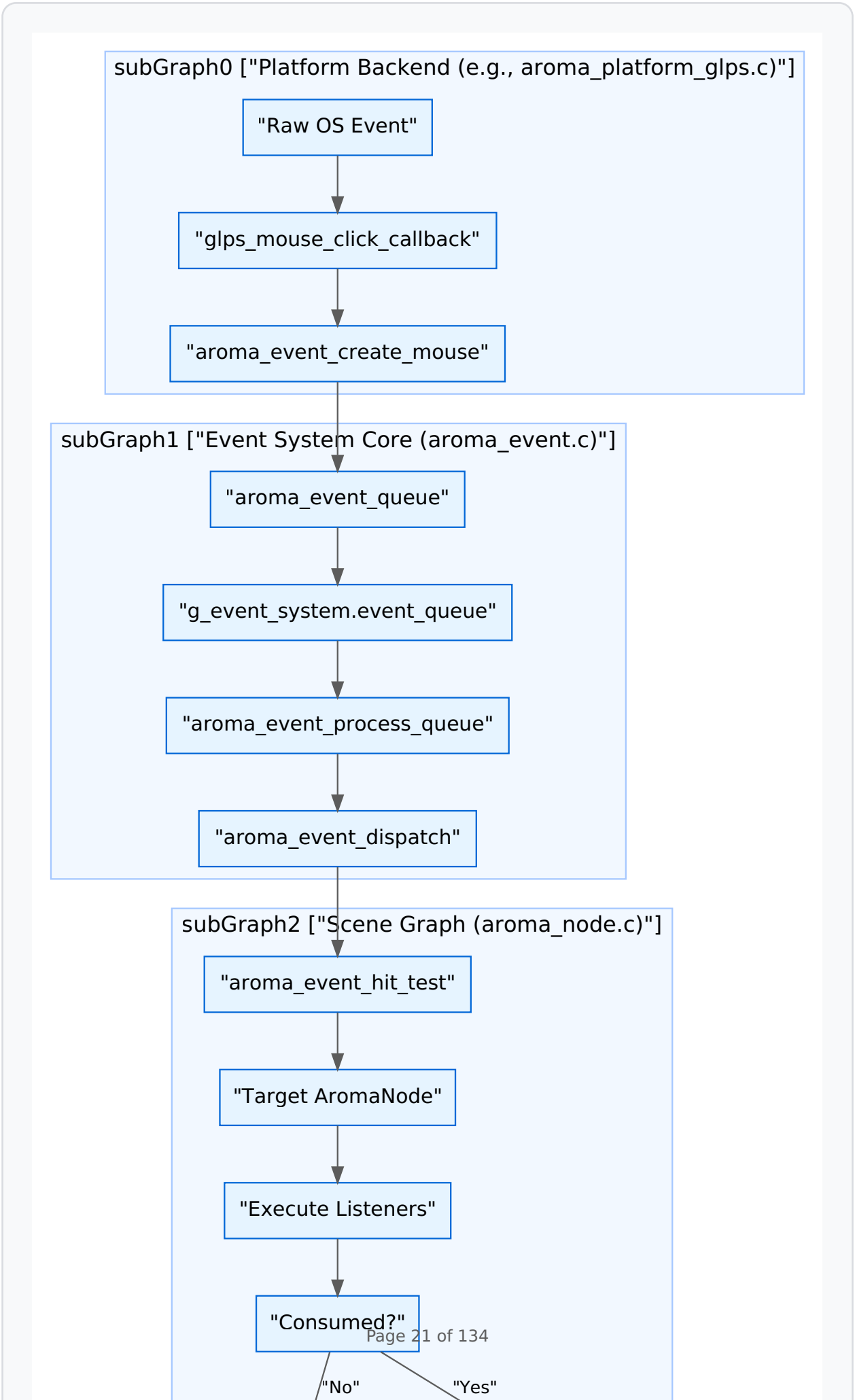
The Event System in AromaUI is a high-performance, deterministic pipeline responsible for capturing raw platform inputs and propagating them through the scene graph. It supports hit-testing, event bubbling, priority-based listeners, and specialized interception for touch-to-scroll physics.

1. Event Pipeline Overview

The event lifecycle follows a strict path from the hardware/OS layer to the individual `AromaNode` handlers.

1. **Capture**: The platform backend (e.g., GLPS, GLFW, Android) captures raw OS events.
2. **Creation**: Events are allocated from a static pool (`g_event_pool`) to avoid runtime fragmentation src/core/aroma_event.c60-63
3. **Hit-Testing**: For pointer events, the system performs a recursive search through the scene graph to find the front-most node at the (x, y) coordinates src/core/aroma_event.c425-460
4. **Queueing**: Events are placed in a thread-safe circular buffer (`event_queue`) src/core/aroma_event.c48-51
5. **Dispatch**: The main loop calls `aroma_event_process_queue()`, which resolves targets and executes listeners src/core/aroma_event.c510-530
6. **Bubbling**: If a listener does not mark an event as `consumed`, it propagates to the parent node src/core/aroma_event.c380-410

Data Flow Diagram: Platform to Node


subGraph0 ["Platform Backend (e.g., aroma_platform_glps.c)"]
"Raw OS Event"
"glps_mouse_click_callback"
"aroma_event_create_mouse"
subGraph1 ["Event System Core (aroma_event.c)"]
"aroma_event_queue"
"g_event_system.event_queue"
"aroma_event_process_queue"
"aroma_event_dispatch"
subGraph2 ["Scene Graph (aroma_node.c)"]
"aroma_event_hit_test"
"Target AromaNode"
"Execute Listeners"
"Consumed?"
"No"
"Yes"

Sources:src/backends/platforms/aroma_platform_glps.c101-114src/core/aroma_event.c510-530src/core/aroma_event.c380-410

---

2. Hit-Testing and Target Resolution

When a pointer event (Mouse/Touch) occurs, the system must identify the target node. This is handled by `aroma_event_hit_test`, which traverses the scene graph starting from the `root_node`src/core/aroma_event.c425-460

- **Z-Order Awareness**: The traversal respects the `z_index` of nodes, ensuring that top-most elements receive events first.
- **Visibility Check**: Nodes with `visible = false` or those outside the parent's clipping rect are ignored.
- **Coordinate Transformation**: Local coordinates are calculated by subtracting the node's absolute position from the screen-space event coordinates.

Sources:src/core/aroma_event.c425-460src/core/aroma_event.h181-182

---

3. Event Listeners and Priority

Nodes do not store listeners directly. Instead, a global hash map (`listener_map`) associates `node_id` with an array of `AromaEventListener` structures src/core/aroma_event.c27-32

Listener Structure

Field	Type	Description
`event_type`	`AromaEventType`	The type of event to filter (e.g., `EVENT_TYPE_MOUSE_CLICK`)
`handler`	`AromaEventHandler`	Function pointer to the callback
`user_data`	`void*`	Context passed to the callback
`priority`	`uint32_t`	Execution order (higher runs first)

Registration API

Users register for events using `aroma_event_subscribe()` src/core/aroma_event.c231-260 When an event is dispatched, the system retrieves the list for that node, sorts them by priority, and executes them until one returns `true` (consumed) src/core/aroma_event.c330-370

Sources:src/core/aroma_event.c27-32src/core/aroma_event.c231-260include/aroma_event.h138-143

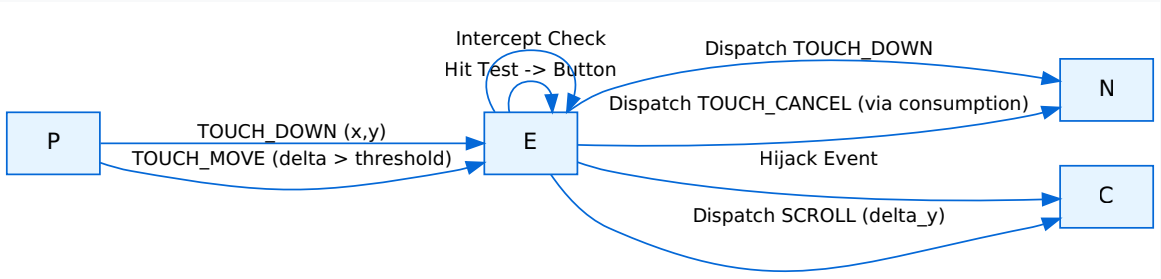
---

4. Touch and Scroll Interception

AromaUI implements a sophisticated "Touch-to-Scroll" interception mechanism to handle scrollable containers (like `AromaListView`) src/core/aroma_event.c74-78

1. **Capture Phase**: On `EVENT_TYPE_TOUCH_DOWN`, the system identifies if the target is within a scrollable ancestor using `find_scrollable_ancestor` src/core/aroma_event.c78
2. **Threshold Detection**: As the user moves their finger (`TOUCH_MOVE`), the system tracks the delta. If the movement exceeds a pixel threshold, the event is "intercepted" by the scrollable container.

3. **Event Hijacking**: Once intercepted, the original target node stops receiving touch events, and they are redirected as `EVENT_TYPE_MOUSE_SCROLL` to the container to drive physics [src/core/aroma_event.c76]



Sources:[src/core/aroma_event.c74-78]()[src/core/aroma_event.c470-500]()

---

5. Memory Management: The Event Pool

To ensure suitability for embedded targets (ESP32/WASM), AromaUI uses a static allocation strategy for events [src/core/aroma_event.c60-63]

- **Static Pool**: `g_event_pool` is a fixed array of 256 `AromaEvent` structures.
- **Free List**: `g_event_free_list` tracks available indices in the pool.
- **Allocation**: `aroma_event_create()` pops an index from the free list [src/core/aroma_event.c265-285]
- **Deallocation**: Once an event is processed or the queue is cleared, the index is pushed back to the free list [src/core/aroma_event.c300-310]

This approach guarantees $O(1)$ allocation/deallocation and zero heap fragmentation during the event loop.

Sources:[src/core/aroma_event.c60-63]()[src/core/aroma_event.c265-285]()

---

6. Key Functions and Entities

Entity	Location	Description
`aroma_event_system_init`	src/core/aroma_event.c146-160	Initializes the listener map and event pool.
`aroma_event_dispatch`	src/core/aroma_event.c330-370	The core logic for running listeners and bubbling events.
`aroma_event_hit_test`	src/core/aroma_event.c425-460	Recursively finds the node at coordinates.
`aroma_event_process_queue`	src/core/aroma_event.c510-530	Called every frame to drain the `event_queue`.
`AromaEvent`	include/aroma_event.h109-125	The primary event container (union of Mouse, Key, Touch, etc.).

Sources:src/core/aroma_event.c146-160src/core/aroma_event.c330-370include/aroma_event.h109-125

Layout Engine

The AromaUI Layout Engine is a top-down recursive system responsible for calculating the screen-space coordinates and dimensions of every node in the scene graph. It operates by combining "self-layout" properties (how a node positions itself relative to its parent) and "container-layout" modes (how a node arranges its children) docs/ui/layouts.md4-9

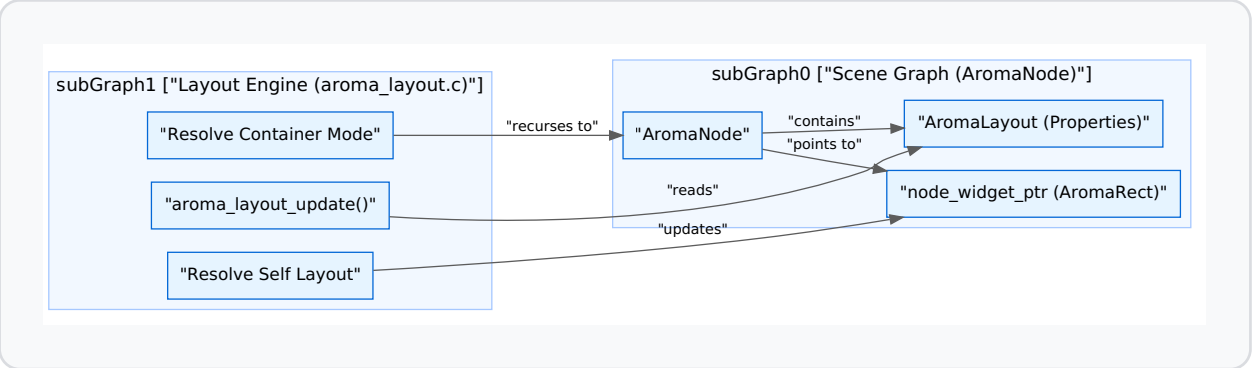
System Overview

Layout is resolved during the frame update phase. The engine traverses the `AromaNode` tree, starting from a root node, and writes final pixel values into the `AromaRect` structure associated with each widget docs/ui/layouts.md8-13 The system uses signed 32-bit integers for coordinates, with the origin `(0, 0)` located at the top-left corner of the parent's bounds docs/ui/layouts.md13

Layout Process Flow

The following diagram illustrates the relationship between the Scene Graph and the Layout Engine logic.

"Layout Execution Flow"



Sources: include/aroma_node.h123-148src/core/aroma_layout.c114-117

Self-Layout Types

Self-layout determines a node's geometry relative to the bounds provided by its parent docs/ui/layouts.md19-21 These types are defined in `AromaLayoutType` include/aroma_node.h43-48

Type	Function	Description
`AROMA_LAYOUT_NONE`	N/A	Absolute positioning. Geometry is left unchanged by the engine docs/ui/layouts.md33-35
`AROMA_LAYOUT_FILL_PARENT`	`aroma_node_set_layout_fill`	Node matches parent bounds exactly (`x=parent_x`, `y=parent_y`, `w=parent_w`, `h=parent_h`) docs/ui/layouts.md41-54
`AROMA_LAYOUT_CENTER`	`aroma_node_set_layout_center`	Centers node within parent. Requires pre-set `width` / `height` docs/ui/layouts.md58-71
`AROMA_LAYOUT_ANCHOR`	`aroma_node_set_layout_anchor`	Pins node to edges with pixel offsets. Supports stretching if both opposite edges are anchored docs/ui/layouts.md75-100

Sources: include/aroma_node.h43-48src/core/aroma_layout.c40-57docs/ui/layouts.md31-115

Container Layout Modes

Container modes determine how a node positions its direct children. This is set via `aroma_node_set_layout_mode` src/core/aroma_layout.c59-61

Flexbox Layout (`AROMA_LAYOUT_MODE_FLEX`)

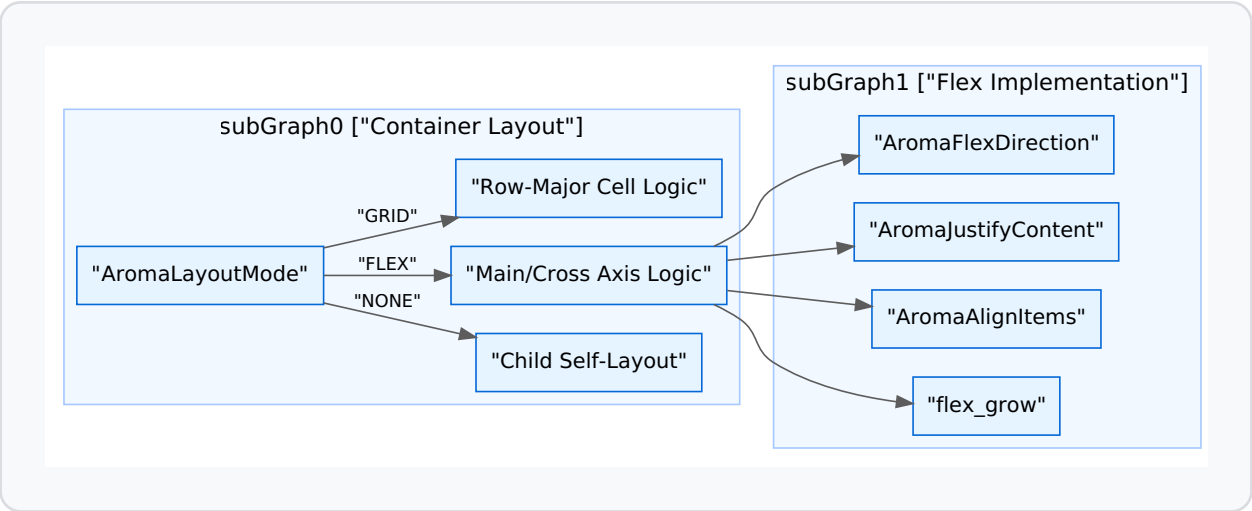
Children are arranged sequentially along a main axis (Row or Column) docs/ui/layouts.md129-135

- **Direction**: `AROMA_FLEX_ROW` or `AROMA_FLEX_COLUMN` include/aroma_node.h58-61
- **Justification**: `justify_content` handles main-axis distribution (Start, Center, End, Space-Between, Space-Around, Space-Evenly) include/aroma_node.h64-71
- **Alignment**: `align_items` handles cross-axis positioning (Start, Center, End, Stretch) include/aroma_node.h74-79
- **Flex Grow**: If a child has `flex_grow > 0`, it absorbs remaining space on the main axis, overriding `justify_content` src/core/aroma_layout.c198-205docs/ui/layouts.md170-172

Grid Layout (`AROMA_LAYOUT_MODE_GRID`)

Children are placed into equal-sized cells in row-major order docs/ui/layouts.md175-184 Cell size is calculated by dividing container dimensions by `grid_cols` and `grid_rows` minus gaps docs/ui/layouts.md189-191

"Container Mode Logic"



Sources: include/aroma_node.h51-80src/core/aroma_layout.c114-117docs/ui/layouts.md117-196

Scrollable Containers and Content Bounds

`AromaContainer` widgets extend the layout engine by managing a viewport that is smaller than its content bounds src/widgets/aroma_container.c113-118

Content Size Calculation

Containers can automatically calculate their content size based on their children using `aroma_container_update_auto_content_size`src/widgets/aroma_container.c117-118 This allows the container to determine the maximum scrollable range:

- `max_scroll_x = content_width - rect.width`src/widgets/aroma_container.c168-172
- `max_scroll_y = content_height - rect.height`src/widgets/aroma_container.c173-177

Physics and Interaction

The container implementation includes a complex interaction model:

1. **Velocity Tracking**: Uses a `VelocityTracker` to sample input speed src/widgets/aroma_container.c47-52
2. **Fling**: Implements kinetic scrolling with friction coefficients and deceleration rates src/widgets/aroma_container.c24-28
3. **Overscroll/Bounce**: Supports stretching beyond bounds with resistance and a spring-back animation src/widgets/aroma_container.c30-32

ListView Integration

`AromaListView` utilizes a specialized scroll container to manage large lists of items src/widgets/aroma_listview.c25-26 It calculates total content height by summing individual item heights (headers, separators, and normal items) and updates the parent container's content size accordingly src/widgets/aroma_listview.c103-119

Sources: src/widgets/aroma_container.c106-163src/widgets/aroma_listview.c103-119

Implementation Details

Data Structures

The primary configuration resides in the `AromaLayout` struct embedded within every `AromaNode`include/aroma_node.h87-118

Internal Alignment

For WebAssembly (WASM) and embedded targets, the `AromaNode` and layout structures include explicit padding to ensure 8-byte alignment, preventing memory access faults include/aroma_node.h144-147

Recursive Update

The layout pass is triggered by `aroma_layout_update` (often called from the main loop). It checks `is_dirty` and `subtree_dirty` flags to skip unnecessary calculations include/aroma_node.h139-142

Sources: include/aroma_node.h87-148src/core/aroma_node.c88-91

Rendering Pipeline & Draw List

The AromaUI rendering pipeline is a multi-stage system designed to transform a hierarchical scene graph into optimized drawing commands. It supports both immediate mode for high-performance desktop rendering and a deferred, batched mode utilizing a `DrawList` for efficient tiling on resource-constrained embedded hardware.

Rendering Lifecycle

The pipeline follows a strict sequence to minimize redundant calculations and GPU state changes.

1. Layout and Invalidation

The process begins when a node's state changes, triggering an invalidation via `aroma_node_invalidate`src/core/aroma_ui_impl.c221 This adds the node to a dirty list. During the next frame update, the system checks if a redraw is necessary using `aroma_ui_consume_redraw`src/core/aroma_ui_impl.c225-229

2. Frame Initialization

A frame is initiated by `aroma_ui_begin_frame`src/core/aroma_ui_impl.c165 This function activates the `AromaDrawList` associated with the target window src/core/aroma_drawlist.c142-145

3. Draw Task Collection and Sorting

Instead of immediate rendering, the system performs a recursive traversal of the `AromaNode` tree to collect `AromaDrawTask` objects src/core/aroma_ui_impl.c71-73

- **Z-Index Sorting**: Tasks are collected into an array and sorted using `qsort` with `draw_task_compare`src/core/aroma_ui_impl.c75 This ensures

that nodes with higher Z-indices are drawn on top regardless of their position in the tree examples/gstreamer_example/ev_cluster.c20-42

- **Frustum Culling**: During collection, nodes are checked against the current clip rectangle; nodes outside the viewport are culled to save processing time.

4. Command Recording

Sorted tasks execute their `draw_cb` include/aroma_drawlist.h30 These callbacks invoke the `AromaBackendABI` proxy functions. If a `DrawList` is active, commands like `aroma_drawlist_cmd_fill_rect` or `aroma_drawlist_cmd_text` are recorded into a command buffer src/core/aroma_drawlist.c174-230

5. Backend Flush

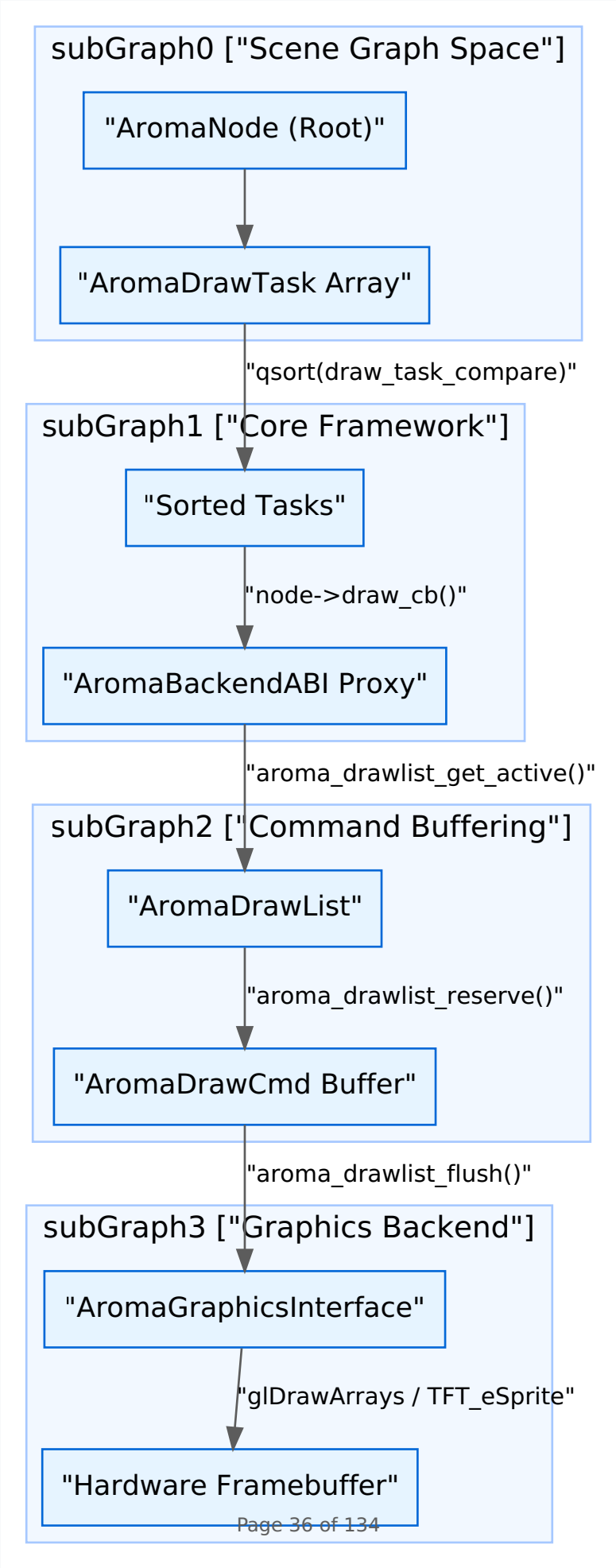
Finally, `aroma_ui_end_frame` src/core/aroma_ui_impl.c166 calls `aroma_drawlist_flush` src/core/aroma_drawlist.h184 which iterates through the buffered commands and dispatches them to the selected `AromaGraphicsInterface` (GLES3, Vulkan, or TFT_eSPI) src/backends/aroma_abi.c13-32

Sources:src/core/aroma_ui_impl.c68-76src/core/aroma_drawlist.c142-160include/aroma_drawlist.h28-46

Data Flow: From Node to Screen

The following diagram illustrates the transition from high-level `AromaNode` objects to the low-level graphics backend via the `DrawList` and `AromaBackendABI`.

Rendering Data Flow


subGraph0 ["Scene Graph Space"]
"AromaNode (Root)"
"AromaDrawTask Array"
"qsort(draw_task_compare)"
subGraph1 ["Core Framework"]
"Sorted Tasks"
"node->draw_cb()"
"AromaBackendABI Proxy"
"aroma_drawlist_get_active()"
subGraph2 ["Command Buffering"]
"AromaDrawList"
"aroma_drawlist_reserve()"
"AromaDrawCmd Buffer"
"aroma_drawlist_flush()"
subGraph3 ["Graphics Backend"]
"AromaGraphicsInterface"
"glDrawArrays / TFT_eSprite"
"Hardware Framebuffer"

Sources:src/core/aroma_ui_impl.c71-75src/core/aroma_drawlist.c87-91src/backends/aroma_abi.c54-58

AromaBackendABI Proxy Pattern

The `AromaBackendABI` acts as a routing layer. It detects if a `DrawList` is active for the current thread. If active, it diverts the call to record a command; otherwise, it passes the call directly to the hardware backend (Immediate Mode).

Proxy Function	DrawList Command	Hardware Backend Fallback
`drawlist_proxy_clear`	`aroma_drawlist_cmd_clear`	`real->clear`
`drawlist_proxy_fill_rectangle`	`aroma_drawlist_cmd_fill_rect`	`real->fill_rectangle`
`drawlist_proxy_render_text`	`aroma_drawlist_cmd_text`	`real->render_text`
`drawlist_proxy_draw_image`	`aroma_drawlist_cmd_image`	`real->draw_image`

Sources:src/backends/aroma_abi.c34-159

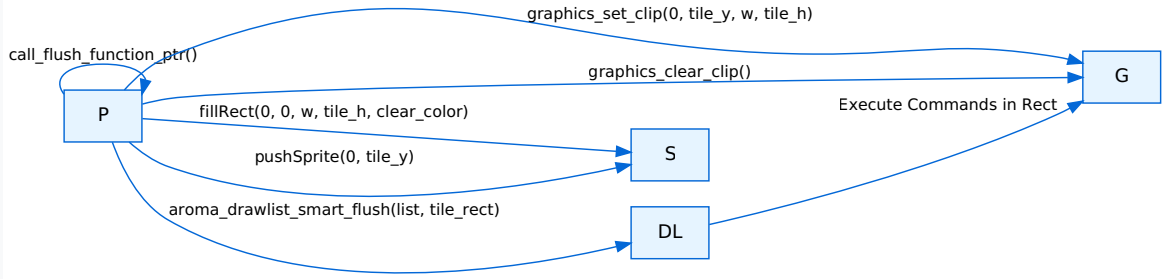
Smart Flushing for TFT Tiling

On embedded targets (e.g., ESP32 with `TFT_eSPI`), memory is insufficient for a full-screen double buffer. AromaUI implements a "Smart Flush" tiling system.

1. **Dirty Tile Tracking**: The screen is divided into horizontal tiles (e.g., `TILE_H = 100`) src/backends/platforms/aroma_platform_tft_espi.cpp21-22

2. **Notification**: When a node is drawn, `tft_mark_tiles_dirty` flags the affected tiles src/backends/platforms/aroma_platform_tft_espi.cpp28-38
3. **Tiled Flush**: `call_flush_function_ptr` iterates only over dirty tiles src/backends/platforms/aroma_platform_tft_espi.cpp169-200
4. **Sprite Batching**: For each dirty tile, the system clears a small `TFT_eSprite`, executes the `DrawList` filtered by a scissor clip matching the tile, and pushes the sprite to the physical display src/backends/platforms/aroma_platform_tft_espi.cpp190-194

Tiling Logic Diagram



Sources:src/backends/platforms/aroma_platform_tft_espi.cpp169-200src/backends/platforms/aroma_platform_interface.h122-135

Batching and Optimizations

GLES3 Shape Batching

The GLES3 backend utilizes a `ShapeBatch` structure to group multiple rectangles into a single draw call src/backends/graphics/aroma_graphics_gles3.c50-56 It buffers up to 256 quads (`MAX_BATCH_QUADS`) before issuing a `glDrawArrays` command src/backends/graphics/aroma_graphics_gles3.c24-26

Font LRU Cache

To manage GPU memory, the GLES3 backend maintains a `CachedFontRenderer` per window src/backends/graphics/aroma_graphics_gles3.c58-64 If the cache exceeds `MAX_FONT_CACHE_PER_WINDOW` (16), the `evict_least_recently_used_font` function removes the least recently used font based on the `current_frame` timestamp src/backends/graphics/aroma_graphics_gles3.c114-144

Sources:src/backends/graphics/aroma_graphics_gles3.c20-95

Scene Graph & Node System

The **Scene Graph** is the foundational structure of AromaUI, representing the visual hierarchy as a tree of `AromaNode` objects. Every element on the screen—from a simple `Label` to a complex `ListView`—is a node within this directed acyclic graph. The system is designed for high performance on embedded hardware through a custom slab allocator, deterministic child limits, and a dual-stage dirty-tracking invalidation system.

The AromaNode: Base Unit

The `AromaNode` is the atomic unit of the UI tree. It encapsulates identity, hierarchy, visibility, and layout properties.

Node Structure and Memory Alignment

To ensure compatibility with WebAssembly (WASM) and strict alignment requirements on certain RISC architectures, the `AromaNode` structure includes explicit padding.

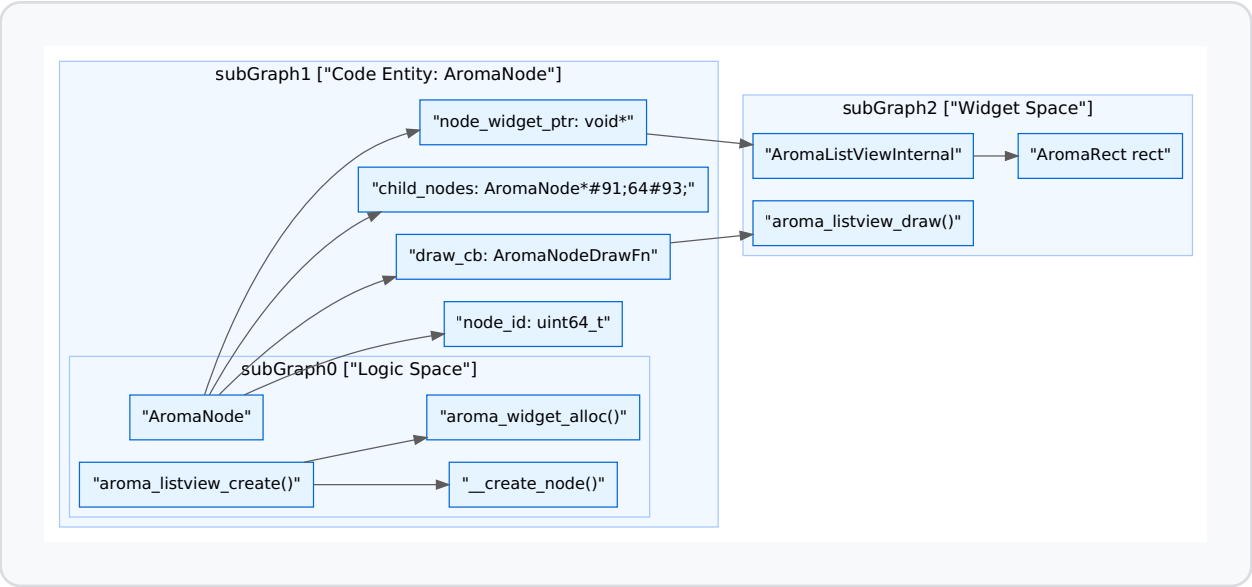
Field	Type	Description
`node_id`	`uint64_t`	A unique identifier generated via `atomic_fetch_add`.
`node_type`	`AromaNodeType`	Categorizes the node as `ROOT`, `CONTAINER`, or `WIDGET`.
`z_index`	`int32_t`	Controls the drawing order; higher values are drawn last (on top).
`node_widget_ptr`	`void*`	Pointer to widget-specific data (e.g., `AromaListViewInternal`).
`_padding`	`uint8_t[4]`	Explicit padding to align the `AromaLayout` struct to an 8-byte boundary.

Sources:include/aroma_node.h123-148src/core/aroma_node.c20-22

Entity Mapping: Node to Logic

The following diagram illustrates how the `AromaNode` structure bridges the gap between the abstract scene graph and concrete widget implementations.

Scene Graph Entity Relationship



Sources:include/aroma_node.h123-148src/widgets/aroma_listview.c53-67src/core/aroma_node.c44-99

Lifecycle Management

Creation and Destruction

Nodes are created via `__create_node` and unlinked from the tree via `__destroy_node`.

- **Child Limit:** A node can have a maximum of **64 direct children** (`AROMA_MAX_CHILD_NODES`). This fixed size allows for deterministic memory layouts and avoids dynamic array reallocations during tree traversal. include/aroma_node.h20
- **Recursive Destruction:** When `__destroy_node` is called, the system first invokes the `destroy_cb` (if present) for widget-specific cleanup, then recursively destroys all children before freeing the node itself. src/core/aroma_node.c153-195

Parent-Child Relationships

The hierarchy is maintained through the `parent_node` pointer and the `child_nodes` array.

- **Addition:** `__add_child_node` handles the allocation and links the child to the parent's array. src/core/aroma_node.c101-123
- **Removal:** `__remove_child_node` performs a linear search for the `node_id`, shifts the array to maintain order, and returns the removed pointer. src/core/aroma_node.c125-151

Dirty-List Invalidation

AromaUI uses a "Dirty-List" system to avoid re-rendering the entire scene graph every frame.

1. **Node Invalidation:** When a property changes (e.g., text, color, position), `aroma_node_invalidate` is called.
2. **State Flags:**
3. `is_dirty`: Indicates the node itself needs a redraw.
4. `subtree_dirty`: Indicates a descendant requires a redraw.
5. **Propagation:** Invalidation propagates the `subtree_dirty` flag upwards to the root, allowing the renderer to skip entire branches of the tree during the draw phase if no flags are set.
6. **Dirty List:** Nodes marked `is_dirty` are added to a global `g_dirty_nodes` array (max 1024) to be processed in the next frame.

Sources:include/aroma_node.h137-143src/core/aroma_node.c17-18

Memory Management: Slab Allocator

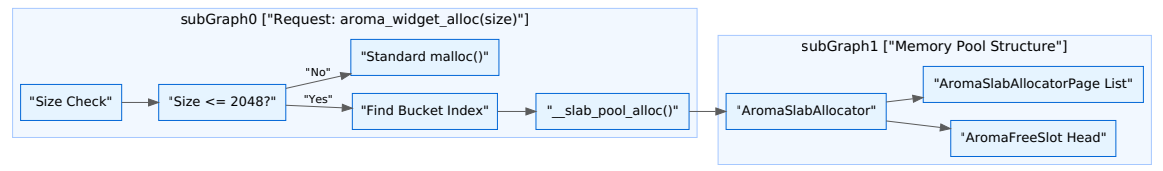
For embedded targets (specifically `ESP32`), AromaUI replaces standard `malloc`/`free` with a high-performance **Slab Allocator** to prevent heap fragmentation.

Global Memory System

The `AromaMemorySystem` manages multiple pools:

- **Node Pool:** Dedicated specifically to `AromaNode` structures.
- **Widget Pools:** Seven buckets ranging from 32 to 2048 bytes for widget-specific data.

Allocator Data Flow



Sources:include/aroma_slab_alloc.h47-64src/core/aroma_node.c66-70

Allocation Logic

1. **Initialization:** `aroma_memory_system_init` preallocates static pages in the `preallocated_pages` array. include/aroma_slab_alloc.h62
2. **Fast Path:** If a `free_list` slot is available, it is popped and returned immediately.
3. **Slow Path:** If no slots exist, a new `AromaSlabAllocatorPage` is carved into objects of the required `object_size`. include/aroma_slab_alloc.h38-54

WASM and Alignment Requirements

AromaUI enforces strict memory alignment to support WebAssembly and various backend architectures.

- **Pointer Validation:** Before accessing a `node_widget_ptr`, the system checks for 4-byte or 8-byte alignment depending on the platform (Emscripten vs. Native). src/core/aroma_layout.c18-38
- **Helper Macros:** The `AROMA_NODE_AS(node, Type)` macro and `aroma_node_get_widget_ptr` function are used to safely cast generic node pointers back to their widget implementations while checking for nulls and alignment. include/aroma_node.h150-160
- **Structure Padding:** The `_padding` field in `AromaNode` ensures that the `AromaLayout` struct starts on an 8-byte boundary, preventing bus errors on sensitive hardware when accessing floating-point flex factors. include/aroma_node.h145-147

Sources:include/aroma_node.h145-160src/core/aroma_layout.c18-38

Theming & Styling

The AromaUI theming system provides a centralized mechanism for managing the visual appearance of applications. It is built around the `AromaTheme` and `AromaStyle` structures, allowing for global configuration of color palettes, spacing, typography, and shadows include/aroma_style.h38-45 The system supports built-in presets (Material, High Contrast, Dark Mode) and granular customization via dedicated utility functions.

Theme System Architecture

The theme system operates at the core framework level, maintaining a global state that widgets consume to resolve their visual properties src/core/aroma_style.c25-26

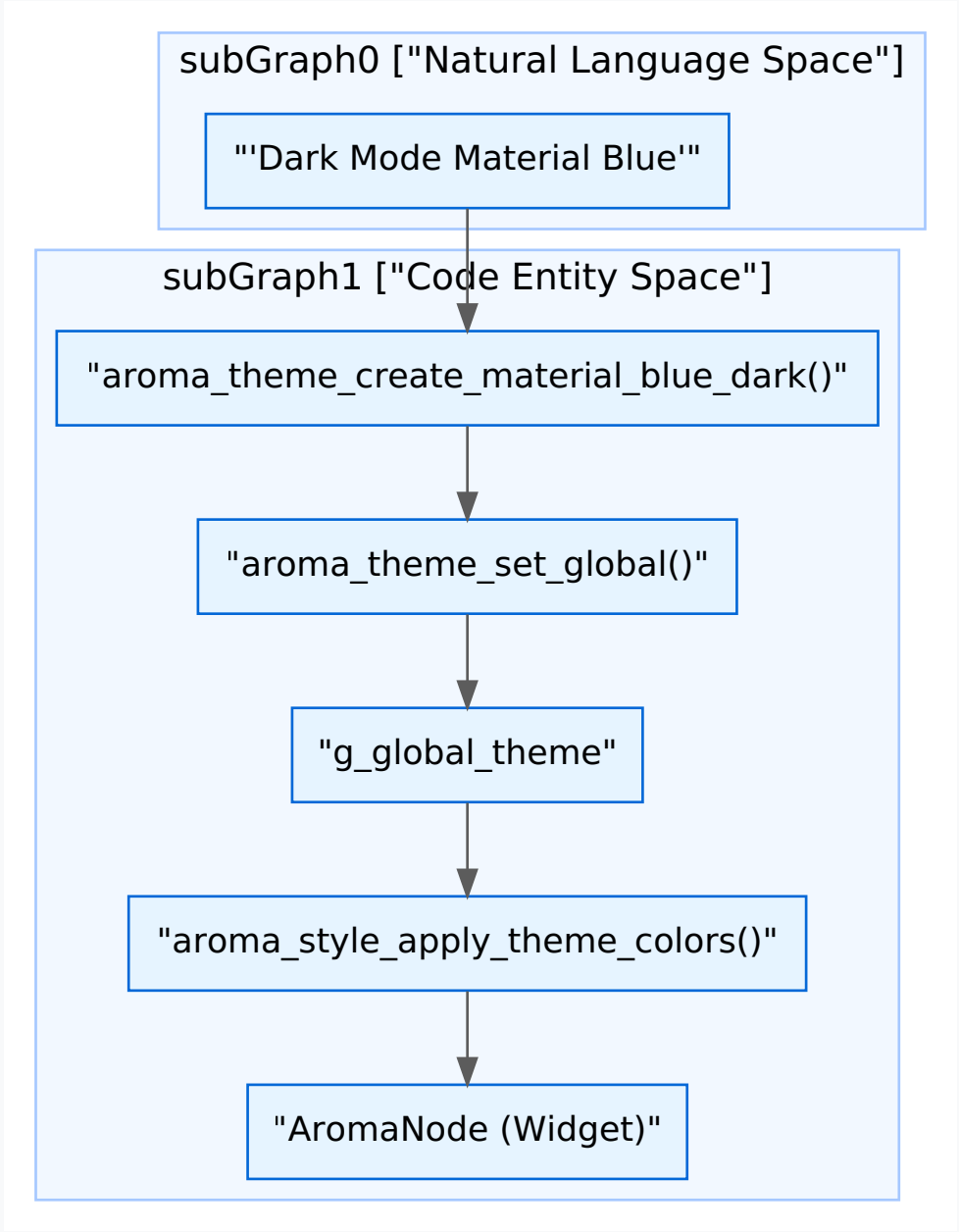
Core Data Structures

Structure	Purpose	Key Fields
`AromaColorPalette`	Defines the color scheme.	`primary`, `background`, `surface`, `text_primary`, `border` include/aroma_style.h9-20
`AromaSpacing`	Defines layout metrics.	`padding`, `margin`, `border_radius`, `border_width` include/aroma_style.h22-28
`AromaTypography`	Defines text appearance.	`font_size`, `line_height`, `font_name`, `font_color` include/aroma_style.h30-36
`AromaTheme`	The top-level theme container.	Combines Palette, Spacing, and Typography include/aroma_style.h38-45
`AromaStyle`	Component-specific styling.	State-based colors (`idle`, `hover`, `active`) and overrides include/aroma_style.h56-78

System Flow Diagram

This diagram illustrates the relationship between theme creation, global state management, and component styling.

Theme Application Pipeline



Sources:include/aroma_style.h100-116src/core/aroma_style.c25-26

Built-in Presets

AromaUI includes several pre-configured themes accessible via factory functions:

- **Default**: A balanced theme based on Material 3 specifications src/core/aroma_style.c73-104

- **High Contrast**: Optimized for accessibility with sharp color boundaries src/core/aroma_style.c138-168
- **Material Presets**: Six color variants (Purple, Blue, Teal, Green, Orange, Pink) in both light and dark modes include/aroma_style.h47-54
- **Black OLED**: Pure black backgrounds (`0x000000`) designed for power efficiency on OLED displays src/core/aroma_style.c202-212

Preset Factory Functions

Function	Description
`aroma_theme_create_default()`	Returns the standard light theme include/aroma_style.h81
`aroma_theme_create_dark()`	Returns the standard dark theme include/aroma_style.h94
`aroma_theme_create_material_preset(preset)`	Returns a Material light theme for a specific `AromaMaterialThemePreset` include/aroma_style.h86
`aroma_theme_create_material_preset_dark(preset)`	Returns a Material dark theme for a specific `AromaMaterialThemePreset` include/aroma_style.h97

Sources:include/aroma_style.h80-104src/core/aroma_style.c73-212

Custom Theme Creation

Developers can create custom themes using `aroma_theme_create_custom()` or by modifying existing presets.

Example: Customizing via theme_manager

In the Car Infotainment example, the `theme_manager.c` demonstrates creating a theme, blending colors, and applying it globally.

```
// Example from theme_manager.c
state.theme = aroma_theme_create_material_blue();
state.theme.enable_shadows = false;
state.theme.colors.background = aroma_color_blend(
    state.theme.colors.primary,
    state.theme.colors.background,
    0.96f
);
aroma_ui_set_theme(&state.theme);
```

Sources:examples/car_infotainment/theme_manager.c14-30

Color Manipulation Utilities

The system provides several functions for dynamic color adjustment, which are useful for generating hover states or blending transitions.

- **`aroma_color_adjust(color, factor)`**: Adjusts brightness. A positive factor lightens the color; negative darkens it src/core/aroma_style.c34-43
- **`aroma_color_blend(color1, color2, blend)`**: Linearly interpolates between two colors based on a factor (0.0 to 1.0) src/core/aroma_style.c45-57
- **`aroma_color_rgb / rgba`**: Packs component bytes into a `uint32_t` (format: `0xAARRGGBB`) src/core/aroma_style.c59-65

Sources:src/core/aroma_style.c34-71

Component Styling

While `AromaTheme` defines the global environment, `AromaStyle` defines how specific widgets respond to that environment.

Applying Theme Colors to Styles

The function `aroma_style_apply_theme_colors(style, theme, is_primary)` maps a theme's palette to a component's state colors (idle, hover, active) include/aroma_style.h116 If `is_primary` is true, the widget will use the theme's primary color as its base; otherwise, it uses secondary/surface colors.

Shadow System

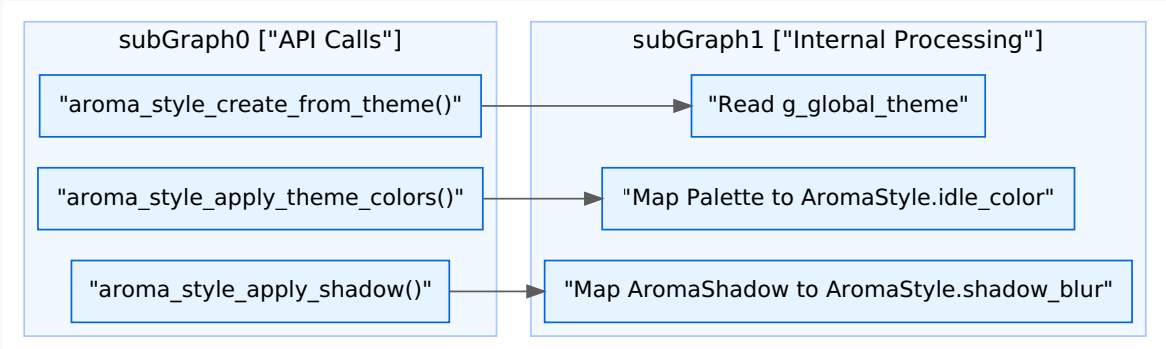
Shadows are managed via the `AromaShadow` struct include/aroma_style.h126-132

- **Presets**: `aroma_shadow_create_soft()`, `aroma_shadow_create_subtle()`, `aroma_shadow_create_deep()` include/aroma_style.h134-137
- **Application**: `aroma_style_apply_shadow(style, shadow)` binds shadow properties to a specific style include/aroma_style.h141

Styling Code Entity Map

This diagram maps the styling functions to their internal logic.

Style Resolution Map



Sources:include/aroma_style.h106-141src/core/aroma_style.c25-26

Display & Feedback Widgets

This page documents the non-interactive and feedback-oriented widgets within the AromaUI library. These widgets are primarily used for information visualization, status reporting, and transient user notifications.

Core Display Widgets

Label and Icon

Labels and Icons are the fundamental building blocks for text and symbolic display. They rely heavily on the `AromaFont` system for rendering.

- **Label**: Displays a single line of text. It uses the `AromaFont` to calculate text metrics like line width and height for proper positioning src/widgets/aroma_label.c1-50
- **Icon**: Renders a single glyph from an icon font (typically Material Symbols). It is essentially a specialized Label where the text is a UTF-8 character code representing an icon src/widgets/aroma_icon.c1-40

Image and GIF

AromaUI supports image rendering from both file paths and memory buffers.

- **Image**: Created via `aroma_image_create` src/widgets/aroma_image.c117-164 or `aroma_image_create_from_memory` src/widgets/aroma_image.c166-200 The widget stores a `texture_id` generated by the graphics backend src/widgets/aroma_image.c34-39
- **GIF**: Handles animated image sequences by maintaining a frame timer and updating the displayed texture periodically src/widgets/aroma_gif.c1-80

Card

The `AromaCard` is a container widget that provides a visual grouping with various styling options.

Card Type	Description
`CARD_TYPE_FILLED`	Uses a solid background color blended with the primary theme color src/widgets/aroma_card.c90-91
`CARD_TYPE_TONAL`	A subtle variant of the filled card.
`CARD_TYPE_GLASS`	Implements a translucent background with a glossy thin edge src/widgets/aroma_card.c101-105
`CARD_TYPE_OUTLINED`	Renders a hollow rectangle with a border src/widgets/aroma_card.c121-127

Data Flow: Card Rendering

1. `aroma_card_create` allocates an `AromaCard` struct and registers it in the `s_card_registry` src/widgets/aroma_card.c130-180
2. During the draw phase, `aroma_card_draw` retrieves the card data using the node's ID src/widgets/aroma_card.c73-75
3. The backend `gfx->fill_rectangle` is called to render the body and shadow src/widgets/aroma_card.c110-119

Sources: src/widgets/aroma_card.c13-24src/widgets/aroma_card.c130-180src/widgets/aroma_image.c34-39

Feedback and Status Widgets

ProgressBar and Gauge

These widgets visualize numerical progress or scalar values.

- **ProgressBar**: A linear bar that fills from left to right based on a 0.0 to 1.0 value.
- **Gauge**: A circular or semi-circular indicator. It calculates vertex positions along an arc to draw the gauge needle and progress track src/widgets/aroma_gauge.c1-100

Loading Spinner

The loading widget provides a continuous animation to indicate background processing. It typically uses the `AromaAnimation` engine to rotate a partial circle or pulse an icon src/widgets/aroma_loading.c1-50

Transient Feedback (Overlays)

Snackbar

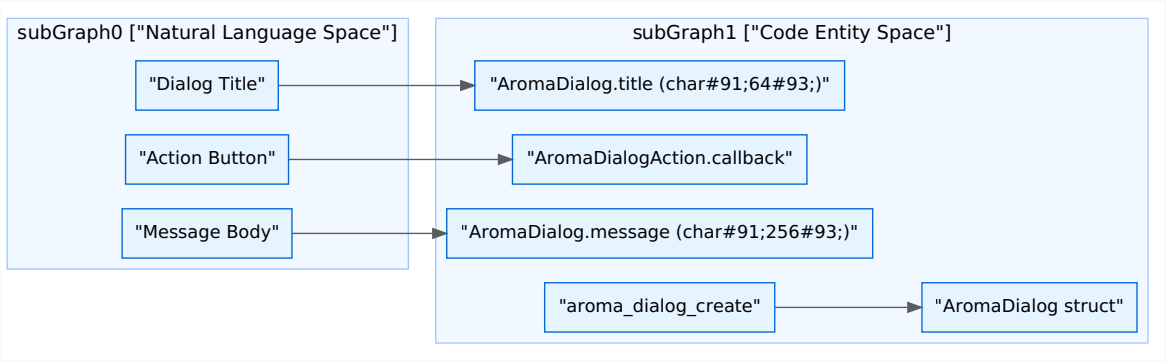
Snackbars provide brief messages at the bottom of the screen. They are designed to automatically dismiss after a set duration.

- **Implementation**: Uses an `AromaTimer` to trigger the `__snackbar_dismiss_cb` src/widgets/aroma_snackbar.c166-176
- **Interaction**: Supports an optional action button (e.g., "UNDO"). Hit testing for the action is calculated based on the font width of the action label src/widgets/aroma_snackbar.c62-70

Dialog

Dialogs are modal windows that interrupt the user. They consist of a title, a message, and up to 3 action buttons src/widgets/aroma_dialog.c13-20

Entity Association: Dialog Component Mapping



Sources: src/widgets/aroma_dialog.c15-47src/widgets/aroma_snackbar.c19-39

Layout Support Widgets

Divider

A simple widget used to separate content. It renders a thin line (horizontal or vertical) using the theme's border color src/widgets/aroma_divider.c1-30

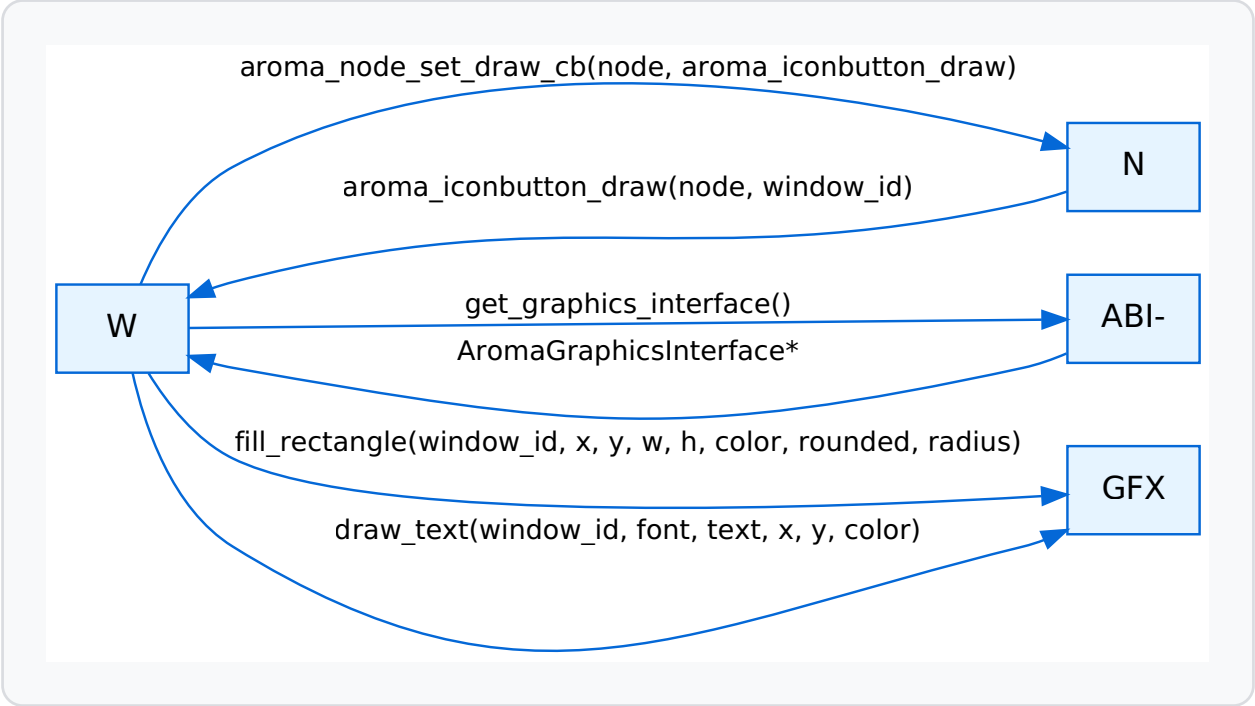
Tooltip

Tooltips are transient labels that appear near a target node. They are typically managed by the global event system, appearing on `EVENT_TYPE_MOUSE_ENTER` after a short delay src/widgets/aroma_tooltip.c1-60

Rendering Integration

Display widgets interface with the `AromaGraphicsInterface` via the `AromaBackendABI`.

Component Interaction: Widget to Backend



Implementation Details:

- **Rounded Rectangles**: Most display widgets (Cards, IconButtons, Snackbars) utilize the backend's ability to render rounded rectangles. In the GLES3 backend, this is implemented via a Signed Distance Field (SDF) shader src/backends/graphics/utils/helpers_gles3.h62-70
- **Z-Index**: Feedback widgets like Snackbars and Dialogs explicitly set a high Z-index (e.g., `INT_MAX`) to ensure they appear above all other UI elements src/widgets/aroma_snackbar.c129

Sources: src/widgets/aroma_iconbutton.c185-210src/widgets/aroma_snackbar.c199-220src/backends/graphics/utils/helpers_gles3.h181-189src/widgets/aroma_card.c66-128

Input & Control Widgets

Input and control widgets form the interactive layer of AromaUI, allowing users to manipulate data and trigger application logic. These widgets are implemented as specialized `AromaNode` objects where the `node_widget_ptr` links to a widget-specific data structure src/widgets/aroma_button.c85

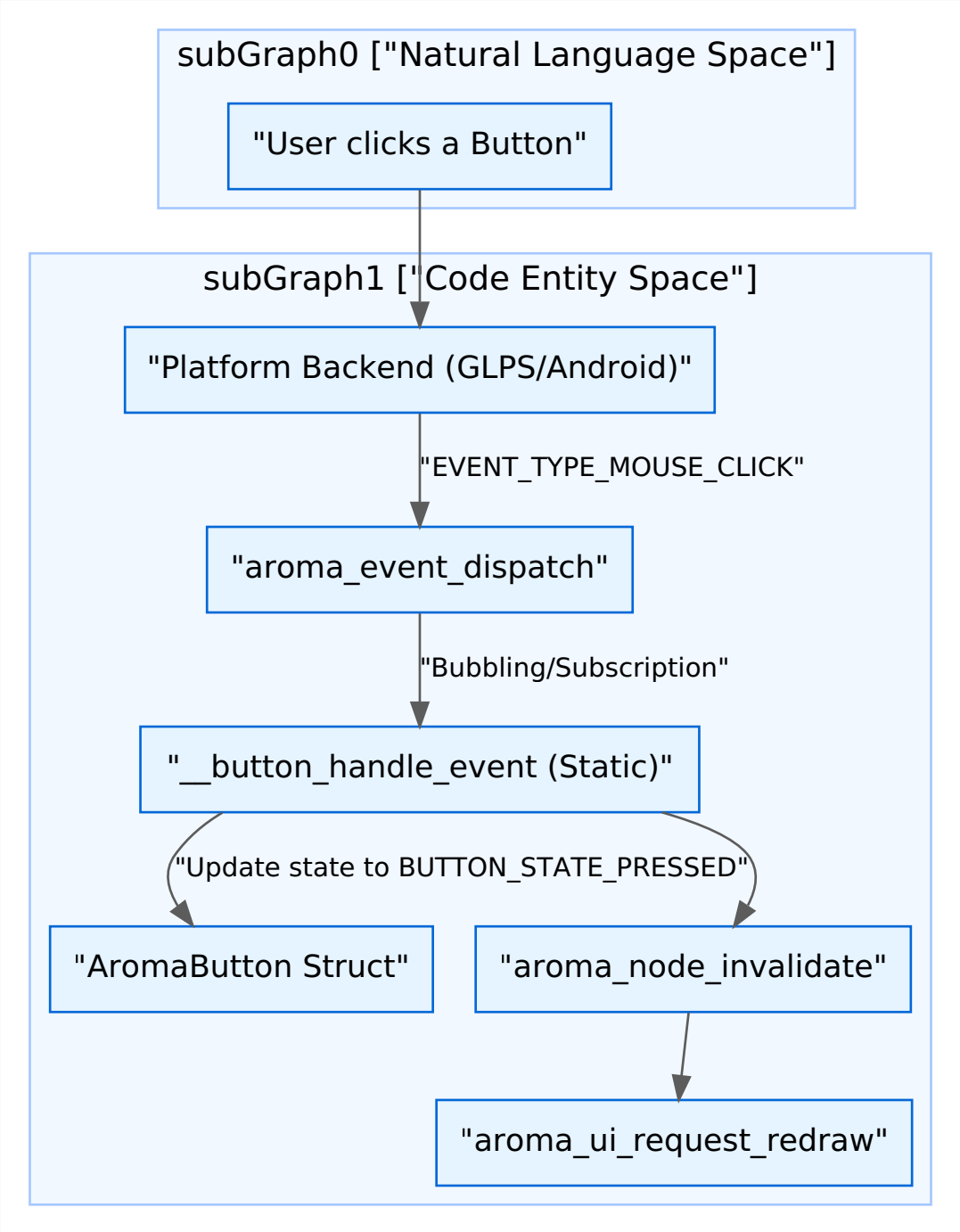
Widget Architecture and Event Flow

Widgets in AromaUI follow a factory pattern for creation and use a subscription-based model for event handling. Most widgets encapsulate their internal state (e.g., `is_pressed`, `is_hovered`) and expose callbacks for application-level logic include/widgets/aroma_button.h44-46

Data Flow: Interaction to Redraw

The following diagram illustrates how a physical interaction (like a mouse click) transitions through the system to update a widget's visual state.

Interaction Pipeline



Sources: [src/widgets/aroma_button.c31-42src/widgets/aroma_button.c139-158src/widgets/aroma_switch.c135-182]

Core Input Widgets

Button and Icon Button

Buttons support labels, icons, and custom color states. They utilize the `AromaTheme` for default styling but allow per-instance overrides src/widgets/aroma_button.c104-109

- **Factory**: `aroma_button_create(parent, label, x, y, w, h)` src/widgets/aroma_button.c70
- **Icons**: Supports prepending an icon using `aroma_button_set_icon` include/widgets/aroma_button.h63
- **Callback**: `bool (*on_click)(AromaNode*, void*)` include/widgets/aroma_button.h44

Checkbox and RadioButton

These widgets manage boolean or mutually exclusive states.

- **Checkbox**: Managed via `aroma_checkbox_set_state` src/widgets/aroma_checkbox.c152
- **RadioButton**: Requires an `AromaRadioGroup` to manage mutual exclusivity among multiple buttons src/widgets/aroma_radiobutton.c17-22
- **Group Logic**: When a button in a group is selected, the group automatically deselects other members src/widgets/aroma_radiobutton.c119-132

Switch and Slider

Used for binary toggles and range-based input.

- **Switch**: Features a sliding thumb animation. State is toggled via `aroma_switch_set_state` src/widgets/aroma_switch.c63

- **Slider**: Maps horizontal coordinates to a value range (`min_value` to `max_value`) src/widgets/aroma_slider.c19-20 It handles dragging events internally to update the `current_value` src/widgets/aroma_slider.c149-156

Textbox

The `AromaTextbox` supports single-line text entry, placeholder text, and cursor management.

- **Focus**: Integration with `aroma_ui_set_focused_node` ensures only one textbox receives keyboard events src/widgets/aroma_textbox.c194
- **Keyboard**: On supported platforms (Android), focusing a textbox triggers `platform->show_keyboard()` src/widgets/aroma_textbox.c195-197
- **Cursor**: Includes logic for cursor blinking and calculating cursor position from mouse clicks src/widgets/aroma_textbox.c62-83

Specialized Controls

Floating Action Button (FAB)

A Material Design inspired button typically used for primary actions.

- **Sizes**: Supports `FAB_SIZE_SMALL`, `NORMAL`, `LARGE`, and `EXTENDED` src/widgets/aroma_fab.c86-94
- **Extended Mode**: Automatically adjusts width based on provided text src/widgets/aroma_fab.c182-184

Chip

Used for filtering or choice selection.

- **Types**: `CHIP_TYPE_ACTION`, `CHIP_TYPE_CHOICE`, `CHIP_TYPE_FILTER`, and `CHIP_TYPE_INPUT` src/widgets/aroma_chip.c15

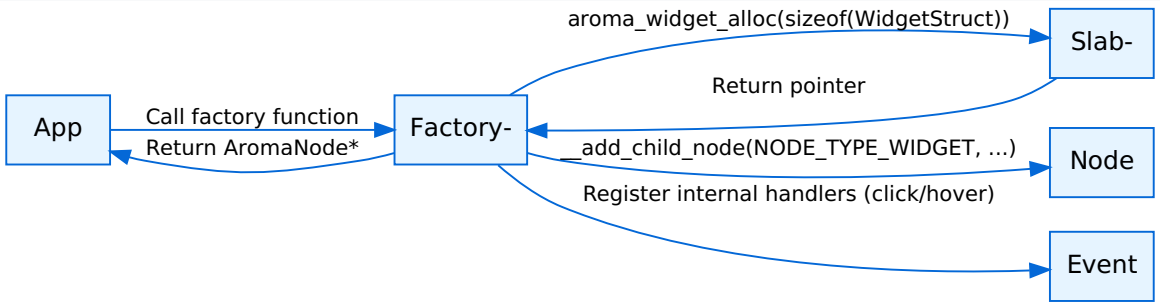
- **Interaction**: Filter chips toggle their `selected` state upon clicking src/widgets/aroma_chip.c80-82

Implementation Summary

Widget Factory and Event Setup

The lifecycle of an input widget involves allocation, node attachment, and event subscription.

Widget Lifecycle Diagram



Sources: src/widgets/aroma_button.c78-93src/widgets/aroma_switch.c19-51src/widgets/aroma_textbox.c92-139

Styling and Themes

Widgets default to the global theme colors but provide explicit setter functions to override appearance.

Widget	Color Properties	Scaling/Sizing
Button	`idle`, `hover`, `pressed`, `text` src/widgets/aroma_button.c180-181	`text_scale`, `corner_radius` include/widgets/aroma_button.h36-38
Slider	`track_color`, `thumb_color` src/widgets/aroma_slider.c43-44	`track_height`, `thumb_size` src/widgets/aroma_slider.c52-54
Textbox	`bg`, `border`, `focused_border` src/widgets/aroma_textbox.c110-118	`text_scale`, `padding_x` src/widgets/aroma_textbox.c16-125
Switch	`color_on`, `color_off` src/widgets/aroma_switch.c31-32	`track_radius`, `toggle_size` src/widgets/aroma_switch.c35-36

Sources: src/widgets/aroma_button.c180-203src/widgets/aroma_slider.c42-56src/widgets/aroma_textbox.c109-131src/widgets/aroma_switch.c30-41

Layout & Navigation Widgets

This page documents the structural and navigational components of AromaUI. These widgets manage the arrangement of child nodes, handle complex scrolling physics, and provide high-level UI patterns like tabbed interfaces, sidebars, and hierarchical lists.

The Container System

The `AromaContainer` is the primary structural widget in AromaUI. It provides a viewport-based clipping region for child nodes and implements a sophisticated scrolling engine.

Scrolling Physics and Viewport

AromaUI uses a "shift subtree" technique for scrolling. Instead of moving the container itself, the `AromaContainer` maintains a scroll offset (`scroll_fx`, `scroll_fy`) which is subtracted from the coordinates of all child nodes during the layout and rendering phases src/widgets/aroma_container.c119-121

Key features include:

- **Velocity Tracking**: The `VelocityTracker` records the last 8 input samples within a 150ms horizon to calculate exit velocity for flings src/widgets/aroma_container.c34-77
- **Fling Physics**: Implements a spline-based deceleration curve that mimics physical friction and gravity src/widgets/aroma_container.c227-246
- **Overscroll and Bounce**: Supports "rubber-band" overscrolling. When a user scrolls past bounds, resistance is applied (`OVERSCROLL_RESIST = 0.35f`). Upon release, a `bouncing` state triggers a 250ms animation back to valid bounds src/widgets/aroma_container.c30-272

Scroll Interception

The container participates in the event pipeline by subscribing to touch events with a specific priority. It intercepts `EVENT_TYPE_TOUCH_DOWN` to stop active flings and monitors `EVENT_TYPE_TOUCH_MOVE` to determine if the movement exceeds `SCROLL_SLOP` (8 pixels) before claiming exclusive focus of the event stream src/widgets/aroma_container.c21-155

Entity Mapping: Container System

Concept	Code Entity	Role
Viewport State	`AromaContainer`	Struct holding offsets and physics state src/widgets/aroma_container.c106-163
Physics Tick	`fling_tick_cb`	Timer callback that updates offsets based on velocity src/widgets/aroma_container.c270
Velocity Logic	`vt_get_velocity`	Computes pixels-per-second from `VTSample` history src/widgets/aroma_container.c75
Layout Sync	`aroma_container_update_auto_content_size`	Recalculates total child area to update scroll bounds src/widgets/aroma_listview.c118

Sources: src/widgets/aroma_container.c17-39src/widgets/aroma_container.c106-163src/widgets/aroma_container.c227-258

ListView and Table

The `ListView` is a specialized container designed for vertical stacks of information. It abstracts the creation of complex rows (icons, primary text, and secondary text) into a simple API.

ListView Implementation

Unlike a raw container, `AromaListView` manages an internal array of `AromaListItem` structures include/widgets/aroma_listview.h12-19

- **Item Types**: Supports `NORMAL`, `HEADER`, and `SEPARATOR` types include/widgets/aroma_listview.h22-24
- **Dynamic Height**: Row heights are calculated dynamically; items with `secondary_text` are rendered at 1.5x the standard height src/widgets/aroma_listview.c90-101
- **Selection Logic**: Hit-testing accounts for the current scroll position of the internal `scroll_container` to map screen coordinates to the correct list index src/widgets/aroma_listview.c121-137

Table Widget

The `AromaTable` widget provides a grid-based display for structured data. It utilizes the `AROMA_LAYOUT_MODE_GRID` layout mode defined in the core node system include/aroma_node.h55

Sources: src/widgets/aroma_listview.c49-53src/widgets/aroma_listview.c90-101include/widgets/aroma_listview.h26-30

Navigation Structures: Tabs and Sidebar

AromaUI provides two primary patterns for top-level navigation: `AromaTabs` (horizontal) and `AromaSidebar` (vertical/responsive).

Tabbed Navigation

The `AromaTabs` widget manages a set of labels and associated `AromaNode` content subtrees.

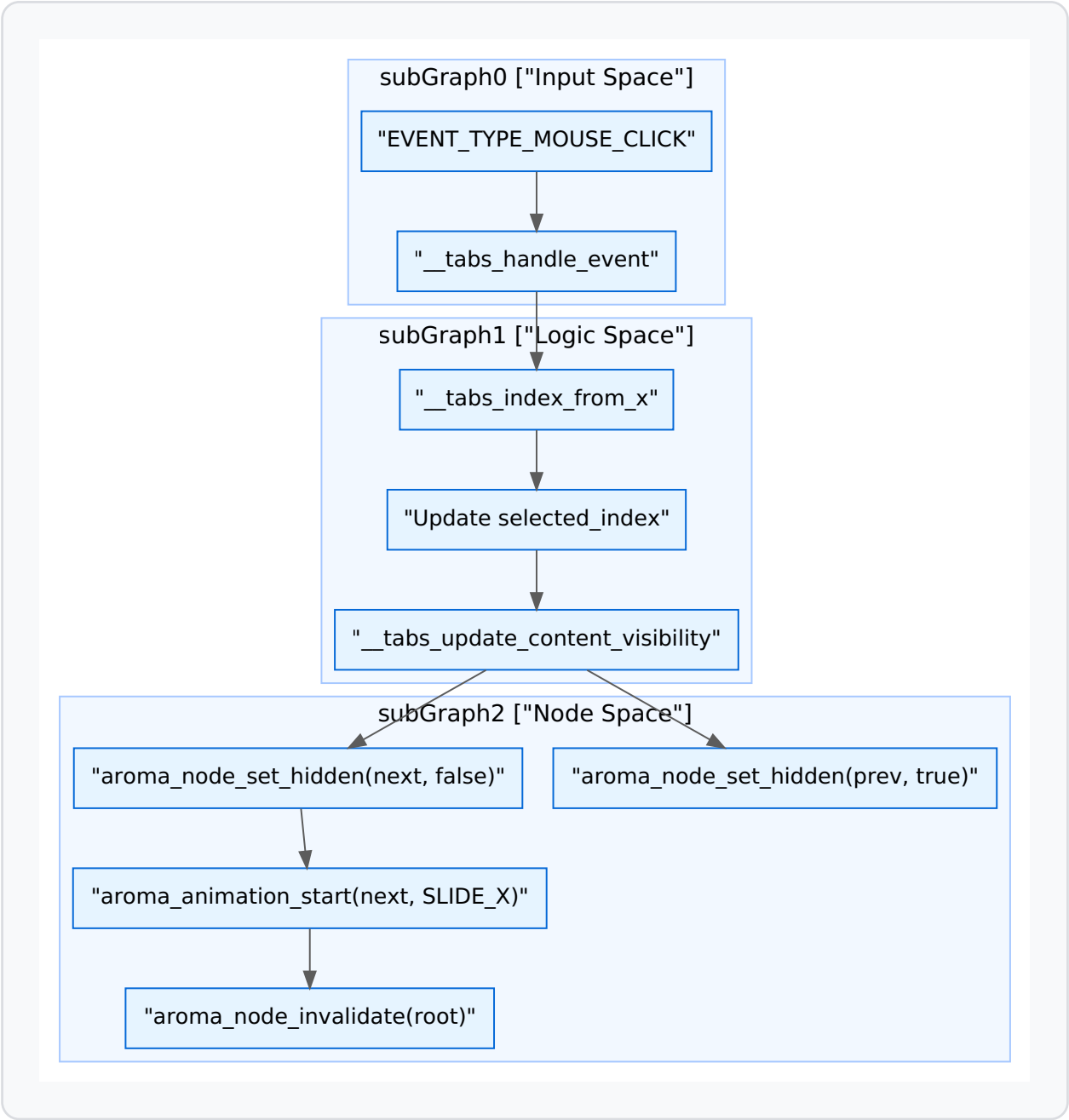
- **Visibility Management**: Only the content nodes associated with the `selected_index` are marked visible. Switching tabs triggers `aroma_node_set_hidden` on the old subtree and removes the hidden flag from the new one src/widgets/aroma_tabs.c92-108
- **Transitions**: Supports animated transitions (`AROMA_ANIM_SLIDE_X`, `AROMA_ANIM_FADE`) when switching between tabs src/widgets/aroma_tabs.c124-133

Sidebar Navigation

The `AromaSidebar` is designed for large-screen or dashboard interfaces (like AromaOS).

- **Responsive Behavior**: It includes a `retracted` state for small screens, switching between `full_width` and `retracted_width` based on a `breakpoint`src/widgets/aroma_sidebar.c55-59
- **Content Swapping**: Similar to Tabs, it manages `content_nodes` for each menu item, ensuring that only the active section is processed by the `collect_draw_tasks` phase of the rendering pipeline src/widgets/aroma_sidebar.c101-112

Data Flow: Navigation Switching



Sources: [src/widgets/aroma_tabs.c19-46src/widgets/aroma_tabs.c92-140src/widgets/aroma_sidebar.c38-70]

Menus, Chips, and Canvas

Context Menus

`AromaMenu` implements a floating overlay. It calculates its height based on the number of items and uses a high-priority event subscription (priority 80) to capture clicks outside its bounds to facilitate "dismiss on click-away" behavior src/widgets/aroma_menu.c34-103

Chips

`AromaChip` is a compact action/filter element. It supports leading icons and trailing "close" buttons, commonly used for filtering lists or representing selected attributes.

Canvas

The `AromaCanvas` widget provides a raw drawing surface. It exposes a `draw_cb` that allows developers to use the `AromaGraphicsInterface` directly to perform custom vector drawing, useful for graphs or custom visualizations include/aroma_node.h33-133

Entity Mapping: Navigation Entities

Code Entity	Function / Role	Source
`aroma_tabs_create`	Factory for horizontal tab bar	src/widgets/aroma_tabs.c182
`aroma_sidebar_create`	Factory for responsive vertical nav	src/widgets/aroma_sidebar.c192
`aroma_menu_add_item`	Appends entry to context menu	src/widgets/aroma_menu.c112
`aroma_node_set_hidden`	Controls subtree rendering traversal	src/core/aroma_node.c1012

Sources: src/widgets/aroma_menu.c34-48src/widgets/aroma_tabs.c182src/widgets/aroma_sidebar.c192

Map Widget

```
AromaNode *map = aroma_ui_map((AromaNode *)window, 0, 0, 700, 400);
if (map)
{
    aroma_map_set_center(map, 33.8869f, 9.5375f);
    aroma_map_add_icon_marker(map, 33.8869f, 9.5375f, 0xFF0000, AROMA_ICON_HOME);
    aroma_map_add_popup_marker(map, 37.7749f, -122.4194f, 0x0000FF, "San Francisco is a cit
    aroma_map_set_route(map, 48.8566, 2.3522, 48.8049, 2.1204, 0xFF35A8FE);
    // aroma_map_set_zoom(map, 12); // Uncomment to set initial zoom level
}
```

The `AromaMap` widget provides a high-performance, interactive mapping component capable of rendering OpenStreetMap (OSM) or CartoDB tiles. It supports complex features such as asynchronous tile fetching with a multi-layered cache, spherical mercator projection, marker/popup overlays, and OSRM-based polyline routing.

Purpose and Scope

The map widget is designed for high-frequency UI updates (60 FPS) on both native (Linux/Android) and web (WebAssembly) targets. It leverages a dedicated background worker pool for networking and image decoding to ensure the main UI thread remains responsive during heavy panning or zooming operations.

Core Architecture and Data Flow

The `AromaMap` widget is built around the `AromaMapExtra` structure, which manages the tile cache, projection state, and overlay data.

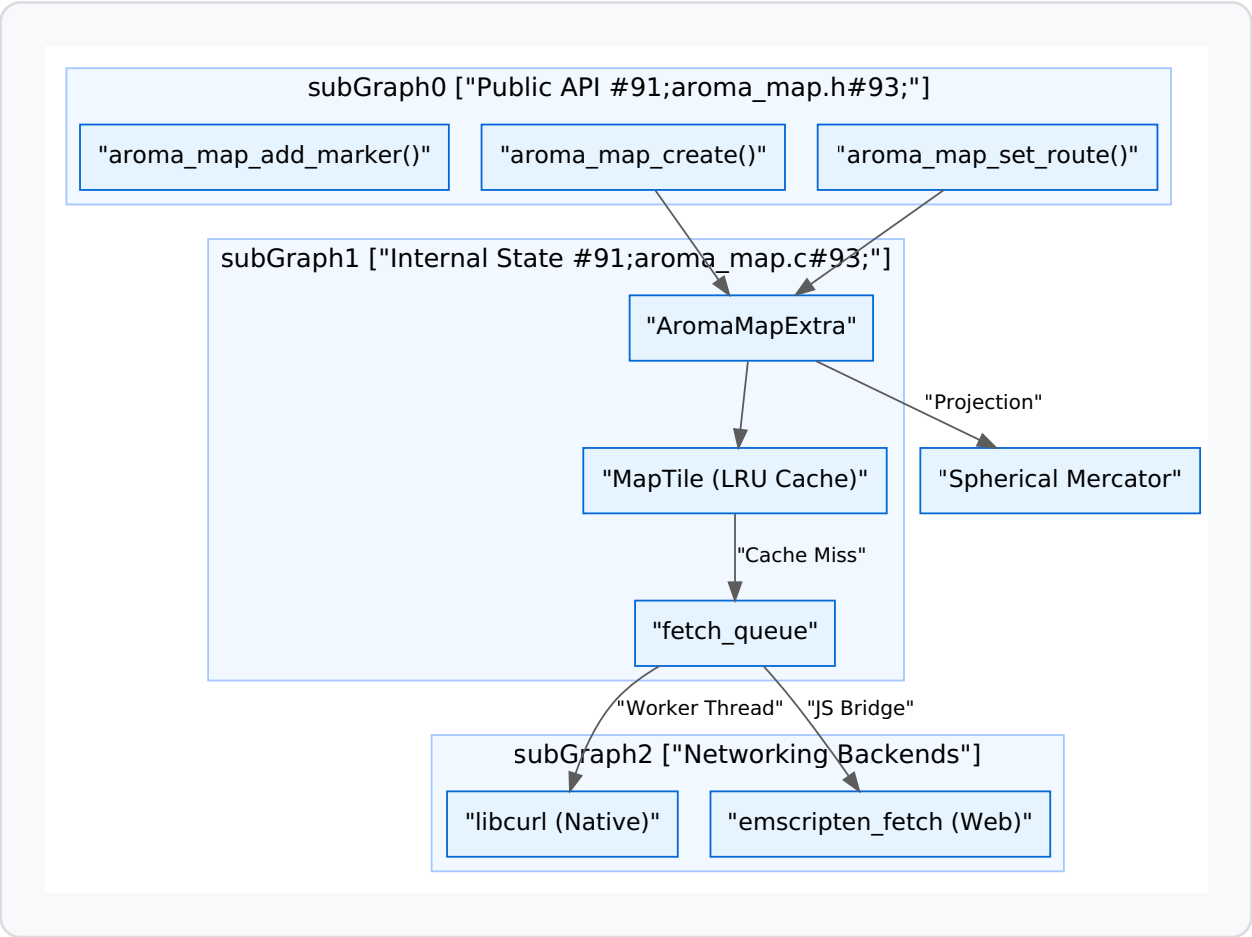
Map Lifecycle

1. **Creation**: `aroma_map_create` allocates the `AromaMapExtra` state and initializes the tile cache and worker threads src/widgets/aroma_map.c73-110
2. **Interaction**: User events (drag, scroll) update the `center_px_x/y` and `zoom` levels include/widgets/aroma_map.h12-27
3. **Tile Fetching**: The widget calculates required tile coordinates (Z, X, Y) and checks the LRU cache. Misses are pushed to a `TileRequest` queue src/widgets/aroma_map.c97-108
4. **Rendering**: The `aroma_map_draw` function projects markers and route polylines from GPS coordinates to screen space and flushes them via the `AromaGraphicsInterface` src/widgets/aroma_map.c187-212

Component Interaction Diagram

The following diagram illustrates the relationship between the high-level widget API and the underlying implementation entities.

"Map Widget Architecture"



Sources:include/widgets/aroma_map.h29-44src/widgets/aroma_map.c68-95src/widgets/aroma_map.c104-130

Tile Management and Fetching

Tiles are 256x256 pixel images fetched from providers like CartoDB or OSM.

LRU Tile Cache

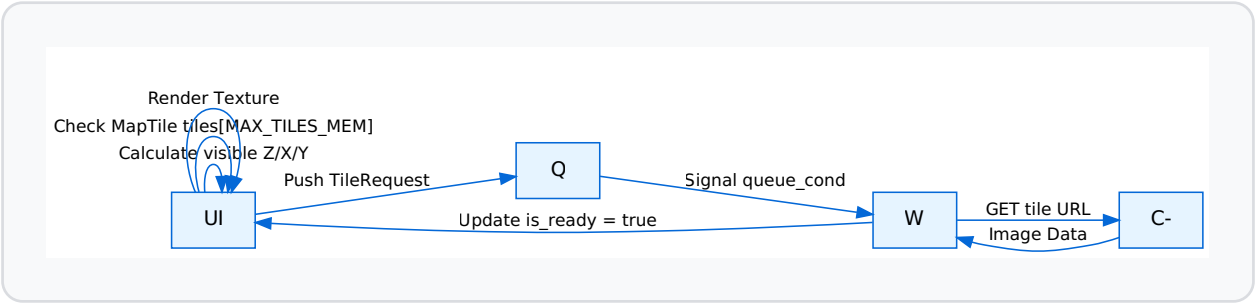
The widget maintains an in-memory cache of `MAX_TILES_MEM` (default 128) tiles src/widgets/aroma_map.c45 Each `MapTile` entry tracks its `access_seq` to implement a Least Recently Used (LRU) eviction policy when the cache is full src/widgets/aroma_map.c48-57

Cross-Platform Fetching

Fetching logic branches based on the target platform:

- **Native**: Uses `libcurl` within a pool of `num_active_workers` (default 2, max 16) src/widgets/aroma_map.c123-128 Tiles are saved to `TILE_CACHE_DIR` (`/tmp/aroma_tiles`) to persist across sessions src/widgets/aroma_map.c44
- **Web (WASM)**: Uses `emscripten_fetch` to leverage the browser's native networking and caching stack src/widgets/aroma_map.c20-22

"Tile Fetching Logic"



Sources:src/widgets/aroma_map.c44-57src/widgets/aroma_map.c104-130src/widgets/aroma_map.c187-212

Projection and Physics

Spherical Mercator Projection

The map uses the standard Web Mercator projection to convert Latitude/Longitude to pixel coordinates. The conversion happens in the routing and drawing phases:

- **Lat to Y**: `(1.0 - log(tan(lat_rad) + 1.0 / cos(lat_rad)) / M_PI) / 2.0` src/widgets/aroma_map.c217
- **Lon to X**: `(lon + 180.0) / 360.0` src/widgets/aroma_map.c218

Panning and Inertia

The widget implements physics-based panning. When a user releases a drag gesture, the `velocity_x` and `velocity_y` parameters in `AromaMapExtra` are used to continue the scroll with deceleration, providing a "fling" effect src/widgets/aroma_map.c82-83

Routing and Overlays

OSRM Polyline Routing

The widget integrates with the Open Source Routing Machine (OSRM).

1. **Request**: `aroma_map_set_route` triggers a background fetch to an OSRM API endpoint include/widgets/aroma_map.h42
2. **Decoding**: The polyline response is decoded from the Google Encoded Polyline Algorithm format into a series of coordinates src/widgets/aroma_map.c157-198
3. **Mutex Safety**: Because route data is fetched in a worker thread and read by the UI thread, access is guarded by `route_mutex`src/widgets/aroma_map.c94

Markers and Popups

Markers are stored in a fixed-size array within the widget state src/widgets/aroma_map.c76

- **Icon Markers**: Rendered using a specific `icon_code` from the icon font src/widgets/aroma_map.c64
- **Popups**: If a marker has `popup_text`, clicking it sets `active_popup_idx`, causing a styled text box to render over the map src/widgets/aroma_map.c65-78

Sources:src/widgets/aroma_map.c157-198src/widgets/aroma_map.c200-237include/widgets/aroma_map.h37-43

Usage Example

The following snippet demonstrates initializing a map with a center point, a marker, and a route between Paris and Versailles.

```
// Initialize Map
AromaNode *map = aroma_ui_map((AromaNode *)window, 0, 0, 1920, 1080);

// Set View
aroma_map_set_center(map, 33.8869f, 9.5375f);

// Add Interactive Elements
aroma_map_add_icon_marker(map, 33.8869f, 9.5375f, 0xFF0000, AROMA_ICON_HOME);
aroma_map_set_route(map, 48.8566, 2.3522, 48.8049, 2.1204, 0xFF35A8FE);
```

Sources:examples/map_example/main.c14-22

Graphics Backends

The Graphics Backend subsystem in AromaUI provides a hardware-agnostic interface for rendering primitives, text, and images. It is managed by the **Backend Abstraction Layer (ABI)**, which routes high-level draw commands to one of three specialized implementations: **GLES3**, **Vulkan**, or **TFT_eSPI**.

Backend Architecture Overview

The `AromaGraphicsInterface` structure defines the contract that every graphics backend must implement. This includes lifecycle management, primitive drawing (rectangles, arcs), text rendering via FreeType, and image loading using `stb_image` or `NanoSVG`.

Graphics Interface Contract

Function Category	Key Functions
Lifecycle	`setup_shared_window_resources`, `setup_separate_window_resources`, `shutdown`
Primitives	`fill_rectangle`, `draw_hollow_rectangle`, `draw_arc`
Text	`render_text`, `measure_text`
Images	`load_image`, `load_image_from_memory`, `draw_image`
State	`graphics_set_clip`, `graphics_clear_clip`, `notify_dirty_region`

Sources:src/backends/graphics/aroma_graphics_interface.h15-107

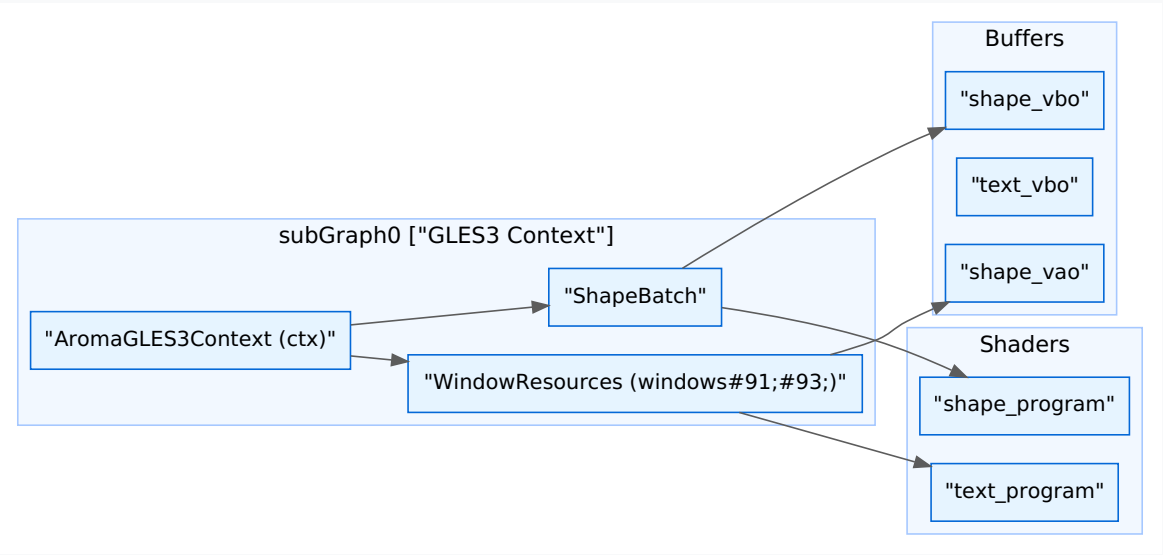
GLES3 Backend

The GLES3 backend is the primary renderer for Desktop (Linux/GLFW) and Android targets. It utilizes modern OpenGL ES 3.0 features to achieve high-performance UI rendering through batching and specialized shaders.

Implementation Details

- **Shape Batching:** To minimize draw calls, the backend uses a `ShapeBatch` structure src/backends/graphics/aroma_graphics_gles3.c50-56 Quads are collected into a vertex buffer (VBO) and flushed in batches of up to 256 quads src/backends/graphics/aroma_graphics_gles3.c24-26
- **Rounded-Rect SDF Shaders:** Rectangles are rendered using a fragment shader (`rectangle_fragment_shader`) that calculates Signed Distance Fields (SDF) to provide perfectly anti-aliased rounded corners and borders without additional geometry src/backends/graphics/aroma_graphics_gles3.c227-230
- **LRU Font Cache:** Each window maintains a `CachedFontRenderer` src/backends/graphics/aroma_graphics_gles3.c59-64 If the cache exceeds `MAX_FONT_CACHE_PER_WINDOW` (16), the Least Recently Used (LRU) font is evicted to save GPU memory src/backends/graphics/aroma_graphics_gles3.c114-144
- **Image Loading:** Integrates `stb_image` for raster formats and `NanoSVG` for vector assets, converting them into GL textures src/backends/graphics/aroma_graphics_gles3.c12-18

GLES3 Entity Mapping



Sources:src/backends/graphics/aroma_graphics_gles3.c28-95

Vulkan Backend

The Vulkan backend provides a high-efficiency alternative for Android and Linux platforms, specifically targeting low-overhead draw call submission and explicit resource management.

Key Features

- **Surface Management:** Uses `vkCreateAndroidSurfaceKHR` (on Android) or KHR surface extensions provided by the platform interface to bind to native windows src/backends/graphics/aroma_graphics_vulkan.c163-174
- **Command Buffering:** Implements a double-buffered frame approach using `VK_MAX_FRAMES_IN_FLIGHT` (2) to allow the CPU to record commands for the next frame while the GPU processes the current one src/backends/graphics/utils/helpers_vulkan.h21

- **Push Constants:** Uses `VkWhiteTexture` and push constants for immediate updates to projection matrices and shape properties (radius, border width) without updating descriptor sets src/backends/graphics/utils/helpers_vulkan.h104-114
- **Text Pipeline:** Uses a dedicated `VulkanTextRenderer` that manages a glyph atlas and specific pipelines for alpha-blended text src/backends/graphics/aroma_graphics_vulkan.c23-29

Sources:src/backends/graphics/aroma_graphics_vulkan.c51-70src/backends/graphics/utils/helpers_vulkan.h62-140

TFT_eSPI Backend (Embedded)

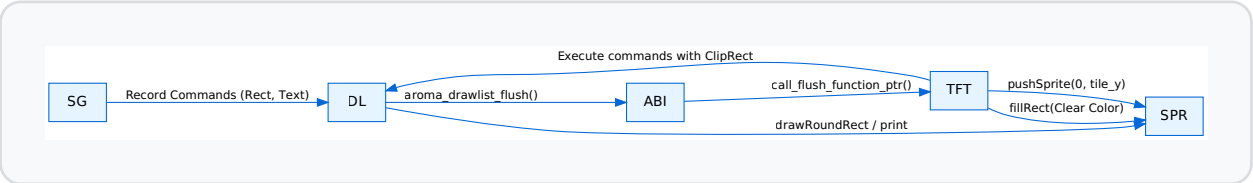
Designed for resource-constrained microcontrollers (e.g., ESP32), this backend interacts with the `TFT_eSPI` library. It emphasizes memory efficiency through tiling and smart flushing.

Tiled Rendering and Double Buffering

Since many embedded targets lack enough RAM for a full-screen frame buffer, AromaUI employs a **Tiled Rendering** strategy:

1. The screen is divided into vertical tiles of height `TILE_H` (typically 100px) src/backends/platforms/aroma_platform_tft_espi.cpp21
2. A `TFT_eSprite` is used as a temporary back-buffer for a single tile src/backends/platforms/aroma_platform_tft_espi.cpp62-68
3. The `call_flush_function_ptr` iterates through dirty tiles, renders the `DrawList` clipped to that tile's bounds, and pushes the sprite to the physical display via SPI src/backends/platforms/aroma_platform_tft_espi.cpp169-200

TFT_eSPI Data Flow



Sources:src/backends/platforms/aroma_platform_tft_espi.cpp28-38src/backends/graphics/aroma_graphics_tft_espi.cpp103-113

The DrawList System

The `AromaDrawList` acts as a command buffer between the UI widgets and the graphics backends. This allows for deferred rendering, Z-index sorting, and the "Smart Flush" mechanism used by the TFT backend.

Command Buffering

Widgets do not call graphics functions directly. Instead, they record `AromaDrawCmd` entries into an active list:

- `aroma_drawlist_cmd_fill_rect` src/core/aroma_drawlist.c174-189
- `aroma_drawlist_cmd_text` src/core/aroma_drawlist.c226-241
- `aroma_drawlist_cmd_image` src/core/aroma_drawlist.c243-256

Smart Flush

The `aroma_drawlist_smart_flush` function is critical for tiled rendering. It accepts a clipping rectangle and only executes commands that intersect with that region, significantly reducing processing time for partial updates include/aroma_drawlist.h186

Sources:src/core/aroma_drawlist.c28-85include/aroma_drawlist.h37-46

Platform Backends

Platform backends in AromaUI serve as the hardware and operating system abstraction layer. They implement the `AromaPlatformInterface` defined in src/backends/platforms/aroma_platform_interface.h35-265 providing standardized hooks for window management, input capture, and system-specific services (e.g., Android Bluetooth or JNI bridges). This architecture allows the Core Framework to remain platform-agnostic while supporting diverse targets ranging from embedded ESP32 displays to desktop Linux and mobile Android environments.

Architecture and Abstraction

The `AromaPlatformInterface` acts as a contract that each backend must fulfill. During initialization, the `AromaBackendABI` routes calls to the active platform implementation src/core/aroma_ui_impl.c170-181

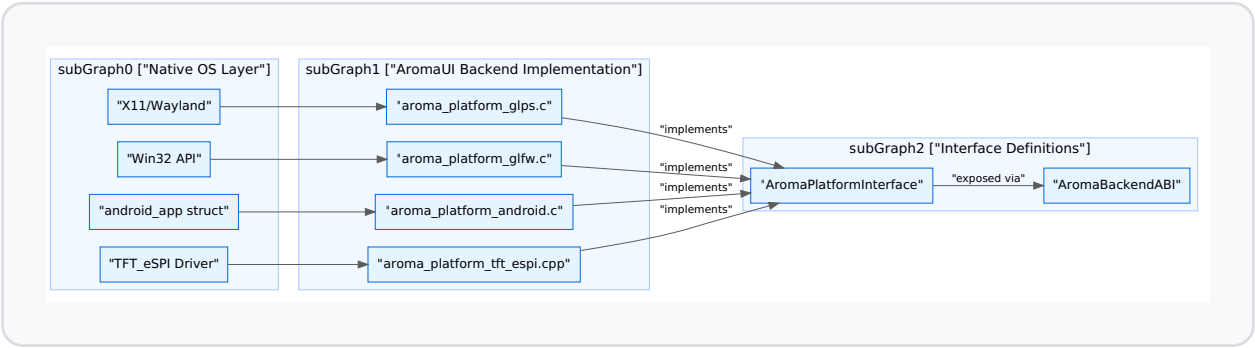
Platform Interface Lifecycle

1. **Initialize**: Sets up the native windowing system or hardware drivers src/backends/platforms/aroma_platform_interface.h43
2. **Window Creation**: Returns a `size_t` window ID used for context management src/backends/platforms/aroma_platform_interface.h62-66
3. **Event Loop**: A blocking or non-blocking call that processes native OS messages and translates them into `AromaEvent` objects src/backends/platforms/aroma_platform_interface.h103
4. **Shutdown**: Releases native resources src/backends/platforms/aroma_platform_interface.h48

Platform-to-Code Mapping

The following diagram maps high-level platform concepts to the specific C structures and functions that implement them.

Platform Backend Entity Mapping



Sources: src/backends/platforms/aroma_platform_interface.h35-265src/backends/platforms/aroma_platform_glps.c31-38src/backends/platforms/aroma_platform_android.c50-58

GLPS (Linux Desktop Default)

The GLPS backend is the primary target for Linux and desktop development. It utilizes the `glps_WindowManager` to handle X11 or Wayland surfaces src/backends/platforms/aroma_platform_glps.c200-205

Input Translation

GLPS captures native events and maps them to the AromaUI event system:

- **Mouse**: `glps_mouse_move_callback` and `glps_mouse_click_callback` calculate deltas and trigger `aroma_event_queue` src/backends/platforms/aroma_platform_glps.c86-114
- **Keyboard**: Translates X11 keycodes to ASCII/UTF-8 values and handles modifiers like `AROMA_KEY_MOD_CAPSLOCK` src/backends/platforms/aroma_platform_glps.c116-173
- **Scrolling**: Maps vertical/horizontal scroll axes to `EVENT_TYPE_MOUSE_SCROLL` src/backends/platforms/aroma_platform_glps.c176-196

Sources: src/backends/platforms/aroma_platform_glps.c1-243

GLFW (Cross-Platform & Headless)

The GLFW backend provides a portable implementation for Windows, macOS, and Linux. It supports both standard windowed modes and a specialized "surfaceless" mode for headless rendering src/backends/platforms/aroma_platform_glfw.c38-51

Surfaceless/EGL Path

For environments without a display server (e.g., GStreamer pipelines), the backend initializes an EGL context using `EGL_KHR_surfaceless_context` src/backends/platforms/aroma_platform_glfw.c63-182

- **Context**: Uses `eglGetDisplay(EGL_DEFAULT_DISPLAY)` and `eglCreateContext` with `EGL_PBUFFER_BIT` src/backends/platforms/aroma_platform_glfw.c103-157
- **Use Case**: Integrated in `examples/gstreamer_example/ev_cluster.c` to render UI frames directly into shared memory (`/aroma_frame_shm`) for video encoding examples/gstreamer_example/ev_cluster.c1-18

Sources: src/backends/platforms/aroma_platform_glfw.c1-219examples/gstreamer_example/ev_cluster.c1-102

Android Backend

The Android backend manages the complex lifecycle of an `ANativeActivity` and coordinates with the Java-side `AromaHelper` via JNI src/backends/platforms/aroma_platform_android.c1-30

Frame Scheduling with AChoreographer

Unlike desktop backends that might poll, Android uses the system `AChoreographer` to sync rendering with the display refresh rate (VSYNC) src/backends/platforms/aroma_platform_android.c80-85

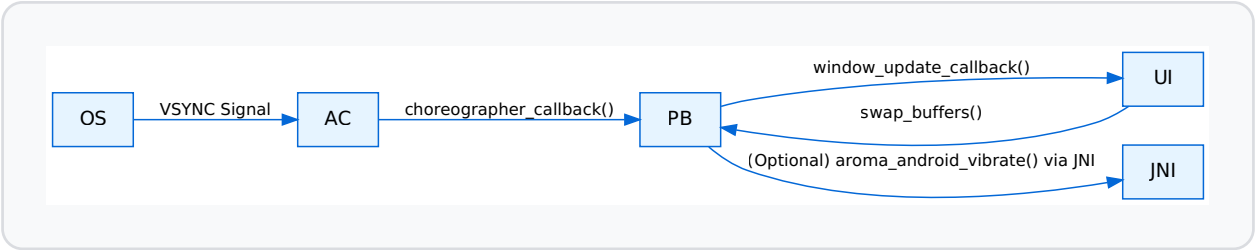
- **`choreographer_callback`**: Triggered by the OS; it calls `update_surface_size` and executes the registered `window_update_callback` src/backends/platforms/aroma_platform_android.c86-114
- **`request_frame`**: Posts a new callback to the choreographer only when a redraw is needed src/backends/platforms/aroma_platform_android.c116-124

Data Flow: JNI and System Services

The backend maintains an `AromaHelperCache` of JNI method IDs to call into Java for system services src/backends/platforms/aroma_platform_android.c133-171

Feature	Implementation Mechanism	Code Reference
Permissions	`platform->android_check_permission` via JNI	include/aroma_android.h89-95
Bluetooth	`android_bt_scan` calling Java `bt_scan`	include/aroma_android.h199-204
Haptics	`android_vibrate` calling Java `vibrator`	include/aroma_android.h136-144
Lifecycle	`android_native_app_glue` events (Focus/Pause)	src/backends/platforms/aroma_platform_android.c4-7

Android Backend Internal Flow



Sources: src/backends/platforms/aroma_platform_android.c86-114include/aroma_android.h136-144

TFT_eSPI and Emscripten

TFT_eSPI (Embedded)

The TFT_eSPI backend targets ESP32 and similar microcontrollers. It is unique in its support for **Tiled Rendering**.

- **Dirty Regions**: It tracks dirty tiles via `tft_mark_tiles_dirty` to minimize SPI bus traffic src/backends/platforms/aroma_platform_interface.h135
- **Flush Mechanism**: The `call_flush_function_ptr` allows the Core to push `AromaDrawList` contents to the `TFT_eSprite` buffer src/backends/platforms/aroma_platform_interface.h122-130

Emscripten (Web)

For WebAssembly targets, the backend utilizes Emscripten's browser hooks.

- **DPI Scaling**: Uses `aroma_emscripten_device_pixel_ratio` to scale the UI for high-density (Retina) displays src/core/aroma_ui_impl.c84-88
- **Main Loop**: Employs `emscripten_set_main_loop` (abstracted in the platform's `run_event_loop`) to prevent blocking the browser's UI thread.

Sources: src/backends/platforms/aroma_platform_interface.h110-141src/core/aroma_ui_impl.c84-102

Project Creation & Scaffolding

The AromaUI toolchain provides a streamlined project initialization system via the `aroma create` command. This system utilizes a template engine to generate cross-platform project structures, handling the complexities of Android directory layouts, JNI setup, and CMake configuration.

The ProjectCreator Engine

The `aroma create` workflow is managed by the `ProjectCreator` class within the Python-based CLI. It automates the transition from a blank directory to a functional AromaUI application by performing placeholder substitution and platform-specific file system organization.

Placeholder Substitution

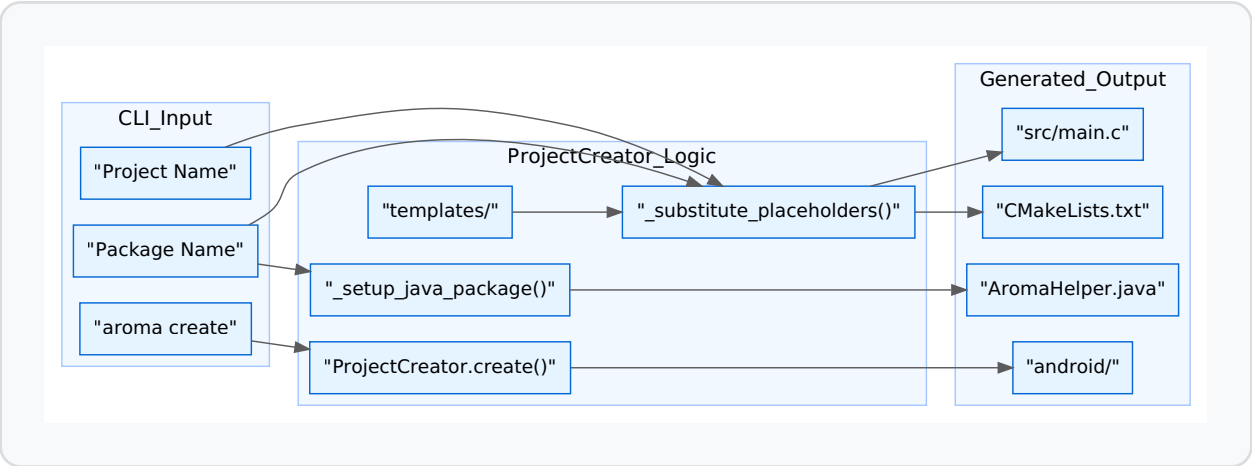
The engine scans template files (typically ending in `.tpl`) and replaces double-curly-brace placeholders with user-provided or system-calculated values:

- `{{PROJECT_NAME}}`: The name of the application tools/cli/templates/android/app/src/main/AndroidManifest.xml6
- `{{PACKAGE_NAME}}`: The Android-style reverse DNS package identifier (e.g., `com.example.app`) tools/cli/templates/android/app/build.gradle6
- `{{AROMA_ROOT}}`: The absolute path to the AromaUI framework source, ensuring the generated project can link against the core library tools/cli/templates/android/app/src/main/cpp/CMakeLists.txt.tpl4

Scaffolding Logic Flow

The following diagram illustrates the transformation from the CLI command to the generated file structure.

Aroma Create Data Flow



Sources: [tools/cli/aroma.py136-141tools/cli/templates/android/app/build.gradle1-15]

Android Scaffolding and Java Package Setup

A significant portion of the scaffolding logic is dedicated to the Android platform. Unlike Linux or Web builds, Android requires a specific directory hierarchy based on the `PACKAGE_NAME`.

_setup_java_package

The `_setup_java_package` function (internal to the CLI) takes a package name like `com.binaryink.test` and:

1. Converts dots to directory separators (`com/binaryink/test`).
2. Creates this path under `app/src/main/java/`.
3. Moves the `AromaHelper.java` template into this deep directory structure [tools/cli/aroma.py124-125]

This ensures that the Android Gradle plugin can correctly locate the Java classes required for the JNI bridge and system service integrations.

Generated Project Layout

A standard project created via `aroma create` contains the following key components:

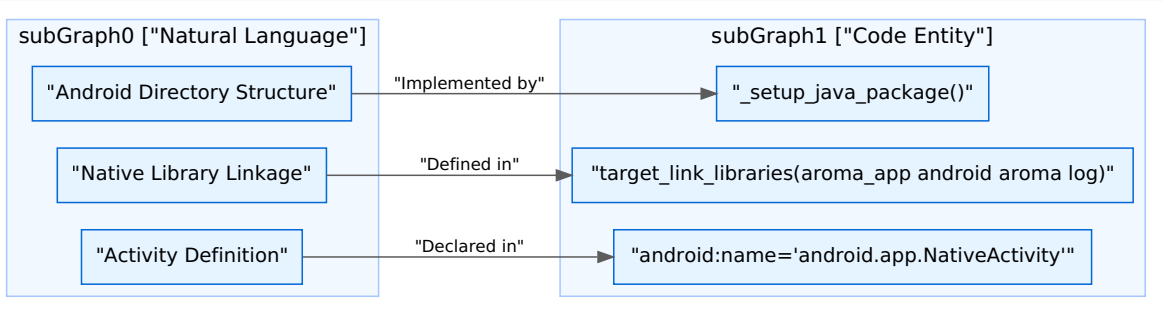
File / Directory	Purpose
`src/main.c`	The application entry point, containing the `main()` and `android_main()` loops tools/cli/templates/app/main.c.tpl29-120
`CMakeLists.txt`	Top-level build script that includes AromaUI and defines the executable.
`android/`	Complete Android Studio compatible project including Gradle wrappers.
`android/app/src/main/AndroidManifest.xml`	Configures the `NativeActivity` and metadata for the native library tools/cli/templates/android/app/src/main/AndroidManifest.xml9-19
`android/app/src/main/cpp/CMakeLists.txt`	Links the app with `android_native_app_glue`, `freetype`, and `aroma_lib` tools/cli/templates/android/app/src/main/cpp/CMakeLists.txt.tpl16-26

The JNI Bridge: AromaHelper.java

The scaffolding includes `AromaHelper.java`, which acts as the primary interface between the C framework and Android System Services. It is responsible for:

- Managing Bluetooth Classic connections and discovery.
- Accessing `SharedPreferences` for persistent storage.
- Handling runtime permissions and hardware features like the Vibrator.

Entity Mapping: Template to Code



Sources: [tools/cli/templates/android/app/src/main/cpp/CMakeLists.txt.tpl26tools/cli/templates/android/app/src/main/AndroidManifest.xml9-14]

Entry Point Implementation

The generated `main.c` demonstrates the standard AromaUI lifecycle. It initializes the UI engine, creates a window, and enters a polling loop that processes events and triggers renders.

```
// tools/cli/templates/app/main.c.tpl:89-93
while (aroma_ui_is_running())
{
    aroma_ui_process_events();
    aroma_ui_render(state.window);
}
```

For Android, the template includes the `android_main` wrapper which bridges the `android_app` state to the AromaUI platform backend:

```
// tools/cli/templates/app/main.c.tpl:115-119
void android_main(struct android_app *state)
{
    aroma_android_set_app(state);
    main(0, NULL);
}
```

Sources: tools/cli/templates/app/main.c.tpl89-93tools/cli/templates/app/main.c.tpl115-119

Building & Deployment

This page documents the complete build and deployment toolchain for AromaUI. The system is managed by the `aroma` CLI, a Python-based utility that abstracts the complexities of CMake, Android Gradle, and Emscripten to provide a unified interface for cross-platform development.

1. CLI Architecture and Environment Validation

The `aroma` CLI is the central entry point for all development tasks. It resides in `tools/cli/aroma.py` and is typically invoked via the `bin/aroma` wrapper script bin/aroma1-9

Environment Validation (`aroma doctor`)

The `doctor` command validates the local development environment. It checks for:

- **System Dependencies**: Python 3, Java (JDK), and Git tools/cli/aroma.py234-240
- **Android SDK/NDK**: Presence of required versions (SDK 34, NDK 25.1.8937393) tools/cli/aroma.py18-29
- **Build Tools**: CMake 3.22.1 and platform-specific compilers tools/cli/aroma.py20

SDK Management (`aroma install-sdk`)

The `AndroidSDK` class automates the acquisition of the Android toolchain. It handles:

1. **Command Line Tools**: Downloads the latest zip from Google repositories tools/cli/aroma.py190-215
2. **License Acceptance**: Automatically accepts Android SDK licenses required for automated builds tools/cli/aroma.py175

3. **Package Installation**: Uses `sdkmanager` to install platforms, build-tools, NDK, and CMake tools/cli/aroma.py23-29

Sources:tools/cli/aroma.py17-36tools/cli/aroma.py144-182bin/aroma1-9

---

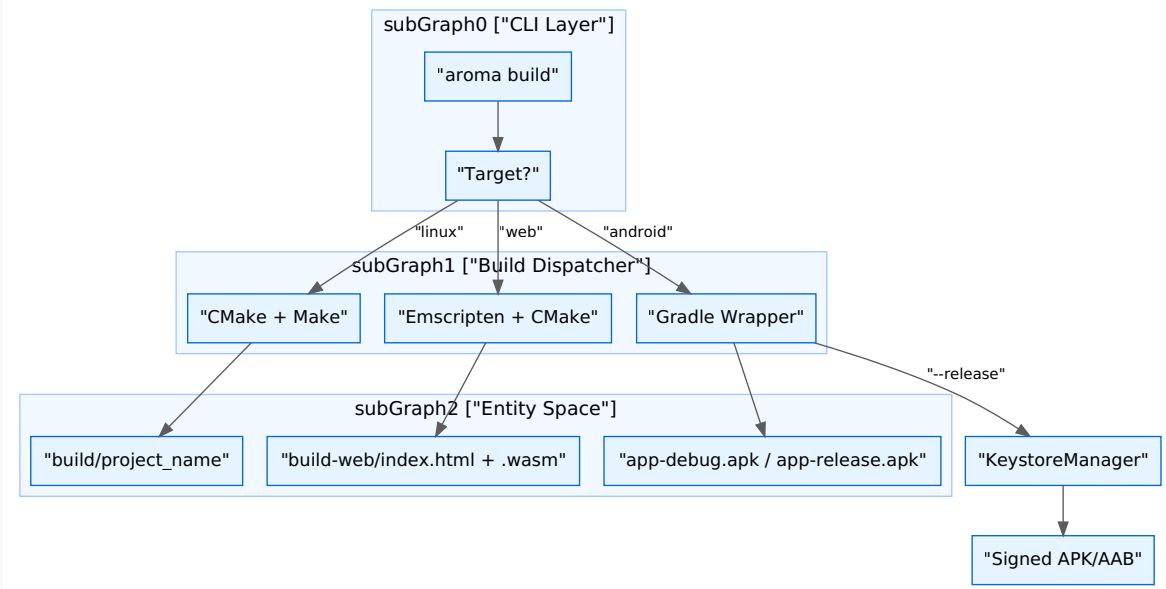
2. Build Process Flow

The build system follows a platform-specific dispatch logic based on the target provided to `aroma build`.

Data Flow: CLI to Compiler

The following diagram illustrates how a build command translates from the CLI into a platform-specific binary.

Build Pipeline Logic



Sources:docs/tools/building_deployment.md22-48docs/tools/building_deployment.md55-67tools/cli/aroma.py144-182

3. Platform Specifics

3.1 Linux Deployment

The Linux build is a native compilation targeting the host architecture.

- **Command**: `aroma build linux`docs/tools/building_deployment.md25
- **Workflow**: The CLI creates a `build/` directory, runs CMake, and executes Make docs/tools/building_deployment.md28-32
- **Execution**: `aroma run linux` launches the binary immediately after ensuring it is up to date docs/tools/building_deployment.md43

3.2 Android Deployment

Android builds utilize a template-based Gradle structure.

- **Debug**: `aroma build android` generates an APK at `android/app/build/outputs/apk/debug/app-debug.apk`docs/tools/building_deployment.md60-66
- **Release**: `aroma build android --release` produces a signed APK docs/tools/building_deployment.md125-131
- **App Bundle**: Adding the `--aab` flag generates the `.aab` format required for Google Play docs/tools/building_deployment.md148-154

3.3 Web (WASM) Deployment

Web builds use the Emscripten toolchain.

- **Command**: `aroma build web`
- **Output**: Files are placed in `build-web/` or `build-ems/` as defined in `.gitignore` .gitignore5-6

Sources:docs/tools/building_deployment.md20-155.gitignore1-6

4. Signing and Security

Release builds for Android require a valid RSA keystore. AromaUI provides a `KeystoreManager` logic within the CLI to simplify this.

Keystore Generation (`aroma sign`)

The `aroma sign` command automates the following:

1. **RSA Key Generation**: Generates a 2048-bit RSA key docs/tools/building_deployment.md103-105
2. **Keystore Storage**: Creates `android/keystore/release.keystore` docs/tools/building_deployment.md109
3. **Properties Mapping**: Generates `android/keystore.properties` which is consumed by Gradle docs/tools/building_deployment.md110
4. **Gradle Integration**: Injects the signing configuration into the `app/build.gradle` template docs/tools/building_deployment.md111

Sources:docs/tools/building_deployment.md95-120tools/cli/templates/android/app/build.gradle1-34

---

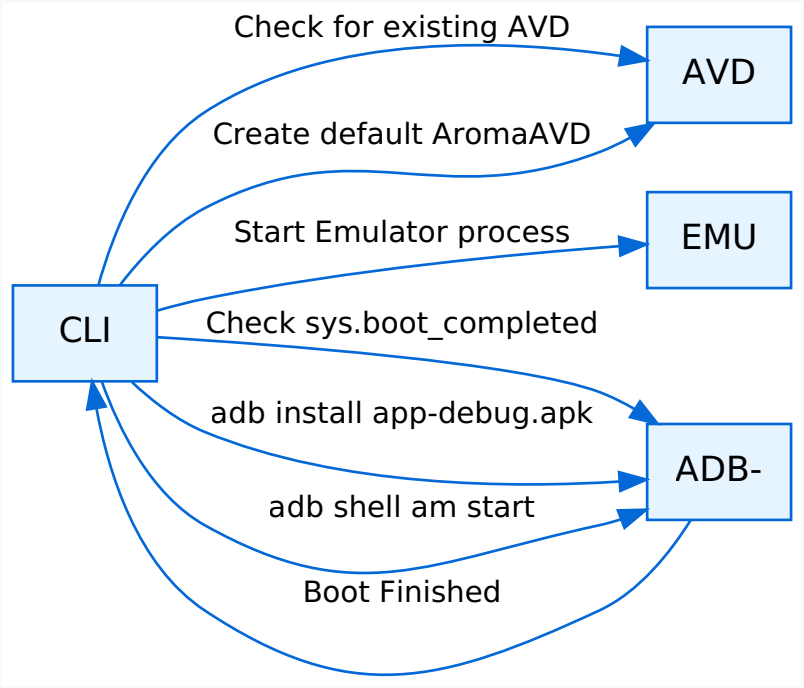
5. Deployment and Emulation

Device Execution (`aroma run`)

The `run` command handles the transfer of binaries to targets:

- **Linux**: Local process execution.
- **Android**: Uses `adb install -r` to push the APK to a connected device docs/tools/building_deployment.md165
- **Emulator**: `aroma run android --emu` triggers the AVD (Android Virtual Device) lifecycle docs/tools/building_deployment.md83

Android Emulator Lifecycle



Sources:docs/tools/building_deployment.md83-93tools/cli/aroma.py151

---

6. Command Summary Table

Command	Action	Key Entities Involved
`aroma build linux`	Native compilation	`CMake`, `Make`
`aroma build android`	Gradle build (Debug)	`gradlew`, `AromaHelper.java`
`aroma build android --release`	Signed Release build	`KeystoreManager`, `gradlew`
`aroma sign`	Keystore setup	`keytool`, `keystore.properties`
`aroma run android --emu`	Emulator deployment	`avdmanager`, `emulator`, `adb`
`aroma install-sdk`	Toolchain setup	`AndroidSDK`, `sdkmanager`
`aroma doctor`	Env validation	`Config`, `subprocess`

Sources:docs/tools/building_deployment.md247-290tools/cli/aroma.py17-29

Car Infotainment (AromaOS)

AromaOS serves as the reference automotive Human-Machine Interface (HMI) for the AromaUI framework. It demonstrates a high-performance, multi-layered dashboard featuring real-time telemetry integration, voice control, and a complex Z-order scene graph suitable for 1280x800 infotainment displays.

Application Lifecycle and Main Loop

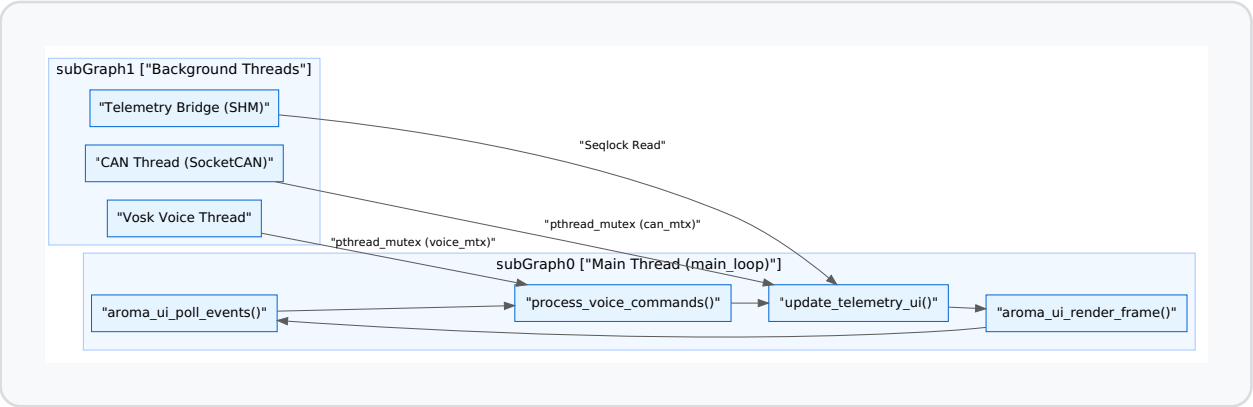
The application entry point in `main.c` orchestrates the initialization of global state, resource managers, and asynchronous processing threads.

Initialization Sequence

1. **Global State**: `init_app_state()` allocates the `AppState` structure and initializes mutexes for thread-safe access to CAN and Voice data examples/car_infotainment/main.c24-28examples/car_infotainment/app_state.c30-66
2. **UI & Fonts**: `aroma_ui_init()` starts the core engine, followed by `init_fonts()` which loads multiple Ubuntu and Icon font instances into the `AppState` for specific UI roles (Clock, Settings, Tabs) examples/car_infotainment/main.c37-52examples/car_infotainment/font_manager.c24-50
3. **Telemetry Bridge**: Attempts to open a POSIX shared memory segment for high-speed vehicle data examples/car_infotainment/main.c68-84
4. **UI Construction**: Builds the hierarchical scene graph by calling `build_vehicle_view`, `build_settings_ui`, and `build_tabs`examples/car_infotainment/main.c85-91
5. **Threads**: Spawns `start_voice_control_thread()` and `start_can_thread()` to handle asynchronous I/O examples/car_infotainment/main.c106-110
6. **Main Loop**: Enters `main_loop()` to process events and render frames examples/car_infotainment/main.c112

Main Loop Diagram

The following diagram illustrates the relationship between the main execution thread and the background data providers.



Sources:examples/car_infotainment/main.c19-125examples/car_infotainment/app_state.h153-155examples/car_infotainment/voice_handler.h20

Telemetry Bridge (Seqlock Shared Memory)

AromaOS utilizes a high-performance telemetry bridge defined in `shared_memory_bridge.h` to ingest data from external vehicle simulators or real hardware.

Seqlock Implementation

The bridge uses a **Seqlock (Sequence Lock)** mechanism to allow lock-free reads from the UI thread while a writer process (like `vehicle_simulator.py`) updates the data.

- **Writer**: Increments `seq_write` to an odd value before writing and an even value after finishing examples/car_infotainment/shared_memory_bridge.h5-9
- **Reader**: `telemetry_bridge_read()` performs a spin-lock check. It reads `seq_write`, performs a `memcpy` of the `telemetry_frame_t`, and then re-

reads `seq_write`. If the values match and are even, the read is considered atomic examples/car_infotainment/shared_memory_bridge.h195-203

Data Structure

The `telemetry_frame_t` carries critical vehicle metrics:

- `speed_kmh_x10`: Speed in km/h (scaled by 10) examples/car_infotainment/shared_memory_bridge.h89
- `battery_soc_pct`: State of Charge percentage examples/car_infotainment/shared_memory_bridge.h96
- `flags`: Bitmask for `FLAG_READY_TO_DRIVE`, `FLAG_LOW_BATTERY`, etc examples/car_infotainment/shared_memory_bridge.h107

Sources:examples/car_infotainment/shared_memory_bridge.h65-112examples/car_infotainment/main.c70-84

Vehicle View and Z-Layer Hierarchy

The primary HMI interface is the `vehicle_view`, which manages a complex stack of visual elements using AromaUI's Z-index system.

Z-Layer Definitions

Layers are defined in `app_state.h` to ensure consistent depth sorting examples/car_infotainment/app_state.h13-25:

Layer Constant	Value	Purpose
`Z_LAYER_BACKGROUND`	1	Road and environmental backgrounds
`Z_LAYER_VEHICLE_IMAGE`	2	The main car sprite (`car.png`)
`Z_LAYER_VEHICLE_OVERLAYS`	3	Dynamic labels (Speed, Range)
`Z_LAYER_STATUS_BAR`	100	Top-level system icons
`Z_LAYER_VOICE_CARD`	999998	Voice assistant modal

Key Components

- **Gear Selector**: Uses a two-card system (`gear_bg_card` and `gear_fg_card`). The foreground card is animated to slide over the active gear letter (P, R, N, D) examples/car_infotainment/vehicle_view.c115-132
- **Interactive Labels**: Includes "Frunk" and "Trunk" labels that act as touch targets for vehicle actuators examples/car_infotainment/vehicle_view.c156-170
- **Battery Diagnostics**: The `battery_diagnostics()` callback triggers a transition that hides standard vehicle info and displays a detailed battery health overlay using `AROMA_ANIM_SLIDE_Y` and `AROMA_ANIM_FADE`examples/car_infotainment/vehicle_view.c26-60

Sources:examples/car_infotainment/vehicle_view.c61-170examples/car_infotainment/app_state.h13-25

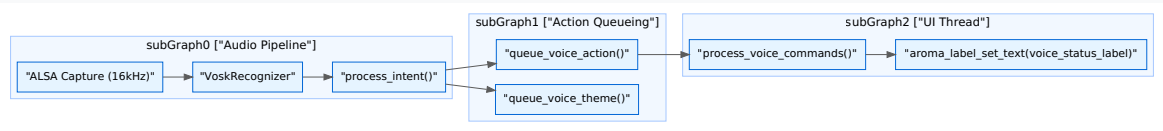
Voice Control System (Vosk Integration)

AromaOS features an integrated voice assistant using the Vosk API for offline speech-to-intent processing.

Intent Processing Pipeline

The voice thread captures audio via ALSA at 16kHz and processes it through `process_intent()` examples/car_infotainment/voice_control.c39-144

1. **Wake Word**: Listens for "hey aroma" or a manual trigger via `trigger_manual_wake()` examples/car_infotainment/voice_control.c40examples/car_infotainment/voice_control.c35-37
2. **Keyword Mapping**: Uses `strstr` to map phrases to UI actions (e.g., "dark mode" calls `queue_voice_theme(1)`) examples/car_infotainment/voice_control.c66-70
3. **UI Feedback**: Commands are queued via `queue_voice_action()`, which the main loop picks up to update `state.voice_status_label` examples/car_infotainment/voice_control.c14-18
4. **TTS**: Uses `pico2wave` and `aplay` to provide audible confirmation examples/car_infotainment/voice_control.c27-33



Sources:examples/car_infotainment/voice_control.c146-160examples/car_infotainment/voice_handler.h14-20

Settings UI and Theme Management

The settings interface demonstrates dynamic layout and theme switching.

UI Structure

- **Slide Animation**: The settings panel is built into `state.settings_panel_node` and is toggled using `AROMA_ANIM_SLIDE_X` for a

smooth transition from the side of the screen examples/car_infotainment/app_state.h30

- **System Info**: Displays hardware telemetry by reading from `/proc` (e.g., CPU usage, memory) and formatting it into AromaUI `ListView` widgets.

Theme Manager

The `theme_manager.c` handles global style shifts. When `state.dark_theme_enabled` is toggled:

1. The `AromaTheme` structure in `state.theme` is updated examples/car_infotainment/app_state.h141
2. `aroma_style_apply_theme_colors()` is called recursively on the root window to update all active nodes.
3. The `backroad` image source is swapped between `backroad_blur.png` and `backroad_dark.png`examples/car_infotainment/vehicle_view.c69-78

Sources:examples/car_infotainment/main.c44examples/car_infotainment/theme_manager.c1-20examples/car_infotainment/app_state.h145

Tabs and Navigation

The application uses a specialized tab manager to switch between the high-level views.

- **Tab Bar**: Located at the bottom of the screen (`WIN_H - 80`), it uses `aroma_ui_tabs_with_icons` to provide navigation between "Vehicle View" and "Settings" examples/car_infotainment/tabs_manager.c6-10
- **Content Switching**: `aroma_tabs_set_content()` associates specific container nodes (like `state.vehicle_view_root`) with tab indices examples/car_infotainment/tabs_manager.c15-16
- **Z-Index**: The tab bar is kept at `Z_LAYER_MAP_BUTTON` (15) to remain visible above the vehicle background but below top-level overlays examples/car_infotainment/tabs_manager.c13

Sources:examples/car_infotainment/tabs_manager.c4-25examples/car_infotainment/app_state.h13-25

Easter Egg: Developer Mode

AromaOS includes a hidden "Developer Mode" triggered by a specific interaction sequence (the "Easter Egg").

- **Trigger**: Usually activated by multiple taps on a specific UI element (e.g., the build info version string).
- **Functionality**: When active, `build_easter_egg_ui()` injects an overlay (`state.easter_egg_overlay`) that provides raw CAN bus logs and advanced rendering statistics examples/car_infotainment/main.c90
- **Implementation**: It utilizes `aroma_node_set_hidden()` to toggle visibility of the debug console without rebuilding the scene graph examples/car_infotainment/app_state.h136-137

Sources:examples/car_infotainment/main.c90examples/car_infotainment/app_state.h136-137examples/car_infotainment/easter_egg.c1-50

Smartwatch & Other Examples

This page documents specialized reference applications and integration examples within the AromaUI ecosystem. These examples demonstrate the framework's versatility across WebAssembly (WASM), headless rendering for multimedia pipelines (GStreamer), and complex widget integrations like the Map system.

WebAssembly Smartwatch Demo

The Smartwatch example is a high-fidelity reference for wearable HMI design. It leverages the Emscripten platform backend to run in modern web browsers while maintaining high performance through a specialized JS bridge and Z-index management.

Implementation Details

The demo utilizes a fixed-size canvas of 380x380 pixels examples/smartwatch_example/main.c14-15 It manages complex UI depth using a hierarchical Z-index system to ensure elements like notifications and app panels appear above the watch face.

Layer Name	Z-Index	Purpose
`Z_WATCH_FACE`	1	Base layer for time and date examples/smartwatch_example/main.c21
`Z_STATS`	2	Activity data (steps, heart rate) examples/smartwatch_example/main.c22
`Z_NOTIFICATION`	10	Overlay for incoming messages examples/smartwatch_example/main.c25
`Z_APPS_PANEL`	15	Full-screen application launcher examples/smartwatch_example/main.c26

JavaScript Bridge and Event Dispatch

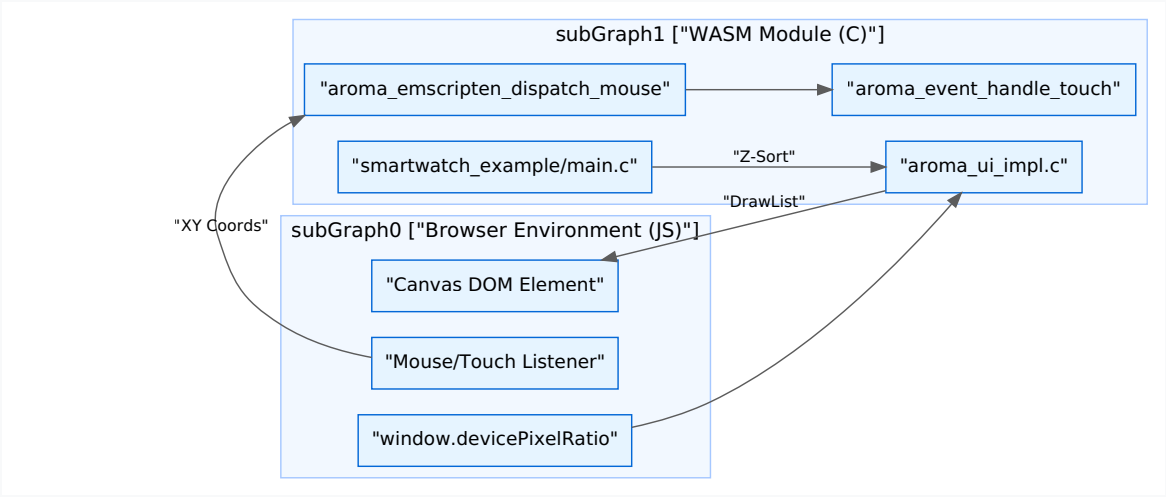
To handle input on the web, AromaUI provides a specialized bridge between the browser's DOM events and the C core.

- **`aroma_emscripten_dispatch_mouse`**: A JS-exported function that translates browser mouse/touch coordinates into the framework's internal event system docs/website/smartwatch_example.js58
- **WASM Heap Management**: The project is compiled with `ALLOW_MEMORY_GROWTH=1` and an initial memory of 128MB to accommodate the font caches and high-resolution assets examples/map_example/CMakeLists.txt43-44
- **DPR Scaling**: The function `aroma_emscripten_device_pixel_ratio` src/core/aroma_ui_impl.c85-88 queries the browser's `window.devicePixelRatio` to ensure crisp rendering on Retina/High-DPI displays.

Smartwatch Component Architecture

The following diagram illustrates the relationship between the C state management and the WASM/JS environment.

WASM Integration and Event Flow



Sources:examples/smartwatch_example/main.c21-28src/core/aroma_ui_impl.c84-88docs/website/smartwatch_example.js58examples/map_example/CMakeLists.txt34-45

Headless Rendering (GStreamer Example)

The GStreamer example demonstrates how to use AromaUI as a graphics overlay engine for video pipelines. This is achieved by rendering the UI into a shared memory buffer without an active windowing system (headless).

Surfaceless EGL

The example uses the `AROMA_USE_EGL_SURFACELESS` flag examples/gstreamer_example/ev_cluster.c1 to initialize the GLFW/GLES3 backend without a native window.

- **Initialization**: The `initialize_egl_surfaceless` function src/backends/platforms/aroma_platform_glfw.c63-182 creates an EGL context using `EGL_PBUFFER_BIT` and the `EGL_KHR_surfaceless_context` extension.

- **Pixel Readback**: Instead of `swap_buffers`, the system can use `aroma_ui_read_pixels` (implemented via `glReadPixels`) to extract the frame into a Shared Memory (SHM) segment examples/gstreamer_example/ev_cluster.c15

Data Flow: Headless Overlay

Sources:examples/gstreamer_example/ev_cluster.c1-18src/backends/platforms/aroma_platform_glfw.c63-172

Map Widget Example

The Map example demonstrates the `AromaMap` widget, which provides a high-performance interactive map with tile-based rendering.

Technical Implementation

- **Tile Fetching**: Uses `libcurl` for native platforms and `emscripten_fetch` for the web examples/map_example/CMakeLists.txt56-67
- **Projection**: Implements Spherical Mercator (EPSG:3857) to map geographic coordinates to pixel space.
- **Integration**: Created via the factory function `aroma_ui_map(parent, x, y, w, h)` include/aroma_ui.h153-158
- **Build Configuration**: Requires Vulkan or GLES3 and specifically links `curl` and `pthread` for asynchronous tile loading examples/map_example/CMakeLists.txt51-69

Sources:include/aroma_ui.h153-158examples/map_example/CMakeLists.txt1-71

Buzzer Quiz-Game App

The Buzzer project serves as a standalone example of an AromaUI consumer. It showcases:

1. **Custom Styling**: Heavy use of `aroma_iconbutton_create` src/widgets/aroma_iconbutton.c89 and `aroma_card_create` to build a game-show aesthetic.
2. **State Synchronization**: Uses the event system to synchronize "buzzer" presses across multiple simulated clients.
3. **Animation Transitions**: Utilizes `aroma_animation_start` examples/smartwatch_example/main.c75 for sliding scoreboards and pulsing feedback icons.

Key Widget: IconButton

The Buzzer app relies on the `AromaIconButton` for its main interface src/widgets/aroma_iconbutton.c13

- **Variants**: Supports `ICON_BUTTON_FILLED`, `ICON_BUTTON_TONAL`, and `ICON_BUTTON_OUTLINED` src/widgets/aroma_iconbutton.c102-107
- **Interactivity**: Implements internal hit-testing and state tracking (`is_hovered`, `is_pressed`) to trigger visual updates via `aroma_node_invalidate` src/widgets/aroma_iconbutton.c46-68

Sources:src/widgets/aroma_iconbutton.c13-138examples/smartwatch_example/main.c71-108

Voice Control System

The AromaUI Voice Control System is a high-performance, offline-capable voice assistant integrated into the AromaOS reference application. It leverages the **Vosk** speech recognition engine and **ALSA** for low-latency audio capture on Linux-based systems. The system provides a natural language interface for vehicle functions such as climate control, media playback, navigation, and system settings.

System Architecture

The voice control system operates in a dedicated background thread to ensure that audio processing and neural network inference do not block the UI rendering pipeline. It communicates with the main UI thread via an asynchronous action queue protected by mutexes.

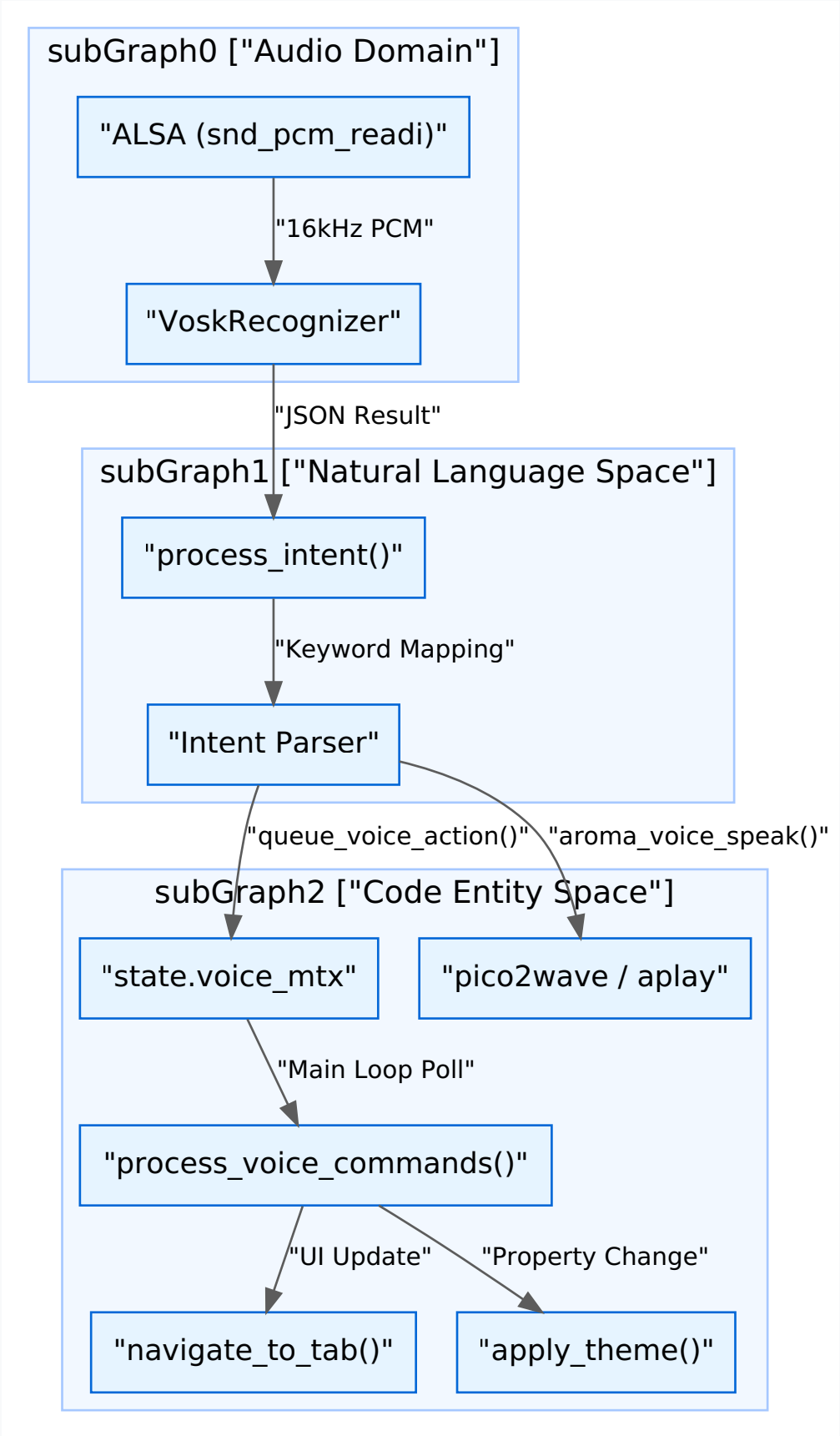
Audio Pipeline and Recognition

Audio is captured via ALSA at a 16kHz sample rate, mono, 16-bit PCM format examples/car_infotainment/voice_control.c146-152 The raw audio buffer is fed into a `VoskRecognizer` which utilizes a pre-trained Kaldi-based model located in the `../model` directory examples/car_infotainment/voice_control.c153-160

Data Flow Diagram

The following diagram illustrates the flow from raw audio capture to UI state updates.

Audio Processing to UI Action Flow


subGraph0 ["Audio Domain"]
"ALSA (snd_pcm_readi)"
"16kHz PCM"
"VoskRecognizer"
"JSON Result"
subGraph1 ["Natural Language Space"]
"process_intent()"
"Keyword Mapping"
"Intent Parser"
"queue_voice_action()"
"aroma_voice_speak()"
subGraph2 ["Code Entity Space"]
"state.voice_mtx"
"pico2wave / aplay"
"Main Loop Poll"
"process_voice_commands()"
"UI Update"
"Property Change"
"navigate_to_tab()"
"apply_theme()"

Sources: examples/car_infotainment/voice_control.c39-144examples/car_infotainment/voice_handler.c170-240examples/car_infotainment/main_loop.c208-211

Wake Word and Activation

The system supports two activation methods:

1. **Wake Word Detection:** The recognizer constantly listens for "hey aroma" or "aroma" examples/car_infotainment/voice_control.c40-41
2. **Manual Wake Window:** A physical or UI button (e.g., the voice icon in the status bar) can call `trigger_manual_wake()`. This sets `manual_wake_time` to the current timestamp examples/car_infotainment/voice_control.c35-37 The system remains in an "active listening" state for a 6-second window, allowing commands to be issued without the wake word examples/car_infotainment/voice_control.c23-25

Intent Parsing and Mapping

The `process_intent` function acts as the primary bridge between recognized text and system logic. It uses keyword matching to map speech to specific functions.

Intent Category	Keywords	Code Entity / Action
Navigation	"navigate to [city]"	`queue_voice_navigation(dest)` examples/car_infotainment/voice_control.c103-111
Climate	"ac up", "hotter", "colder"	`queue_voice_ac_action(delta)` examples/car_infotainment/voice_control.c71-78
Media	"play music", "pause song"	`queue_voice_action(-1, ...)` examples/car_infotainment/voice_control.c44-50
Theming	"dark mode", "light theme"	`queue_voice_theme(1/0)` examples/car_infotainment/voice_control.c61-70
App Switching	"settings", "phone", "home"	`queue_voice_action(tab_index, ...)` examples/car_infotainment/voice_control.c91-102

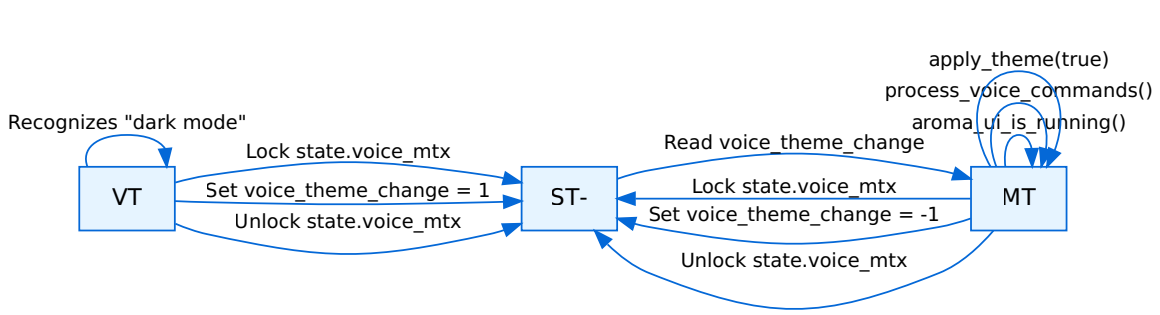
Implementation Detail: Navigation Parsing

When "navigate to" is detected, the system extracts the destination string by offsetting the pointer examples/car_infotainment/voice_control.c104-105 This string is passed to `queue_voice_navigation`, which later performs a lookup against a `CityMap` (containing coordinates for Paris, London, New York, etc.) during the UI thread's processing phase examples/car_infotainment/voice_handler.c179-203

UI Thread Synchronization

Voice actions are queued into the global `state` object using thread-safe "queue" functions. These functions lock `state.voice_mtx` before modifying command flags examples/car_infotainment/voice_handler.c112-126

Voice Action Synchronization



Sources: examples/car_infotainment/voice_control.c66-70examples/car_infotainment/voice_handler.c88-94examples/car_infotainment/voice_handler.c233-240

Text-to-Speech (TTS)

Feedback is provided via the `aroma_voice_speak` function. It utilizes `pico2wave` to generate a temporary WAV file and `aplay` for playback examples/car_infotainment/voice_control.c27-33

- **Command:** `pico2wave -w /tmp/aroma_tts.wav "%s" && aplay -q /tmp/aroma_tts.wav &`
- The process is spawned as a background shell command to prevent audio playback from stalling the voice recognition thread examples/car_infotainment/voice_control.c30

Visual Feedback (Voice Status UI)

The UI provides real-time feedback of the voice assistant's state through a "Voice Status Card."

- **Partial Results:** As the user speaks, `queue_voice_partial` updates the UI with intermediate transcriptions examples/car_infotainment/voice_handler.c74-86
- **Animations:** When the assistant activates, the `voice_status_card` slides into view from the top of the screen (`AROMA_ANIM_SLIDE_Y`) and a `loading_spinner` is revealed examples/car_infotainment/voice_handler.c45-52
- **Timeouts:** If no command is finalized, the status label clears after a timeout (approx. 180 frames) examples/car_infotainment/voice_handler.c221-230

Sources: examples/car_infotainment/voice_handler.c31-61examples/car_infotainment/voice_handler.c217-230

Font System

Font System

The **Font System** in AromaUI provides a unified interface for loading, managing, and rendering typography across diverse platforms, ranging from high-performance desktop environments to resource-constrained embedded systems. It leverages **FreeType** for vector font rasterization on most platforms while providing specialized optimizations for GLES3 backends and fallback mechanisms for TFT-based hardware.

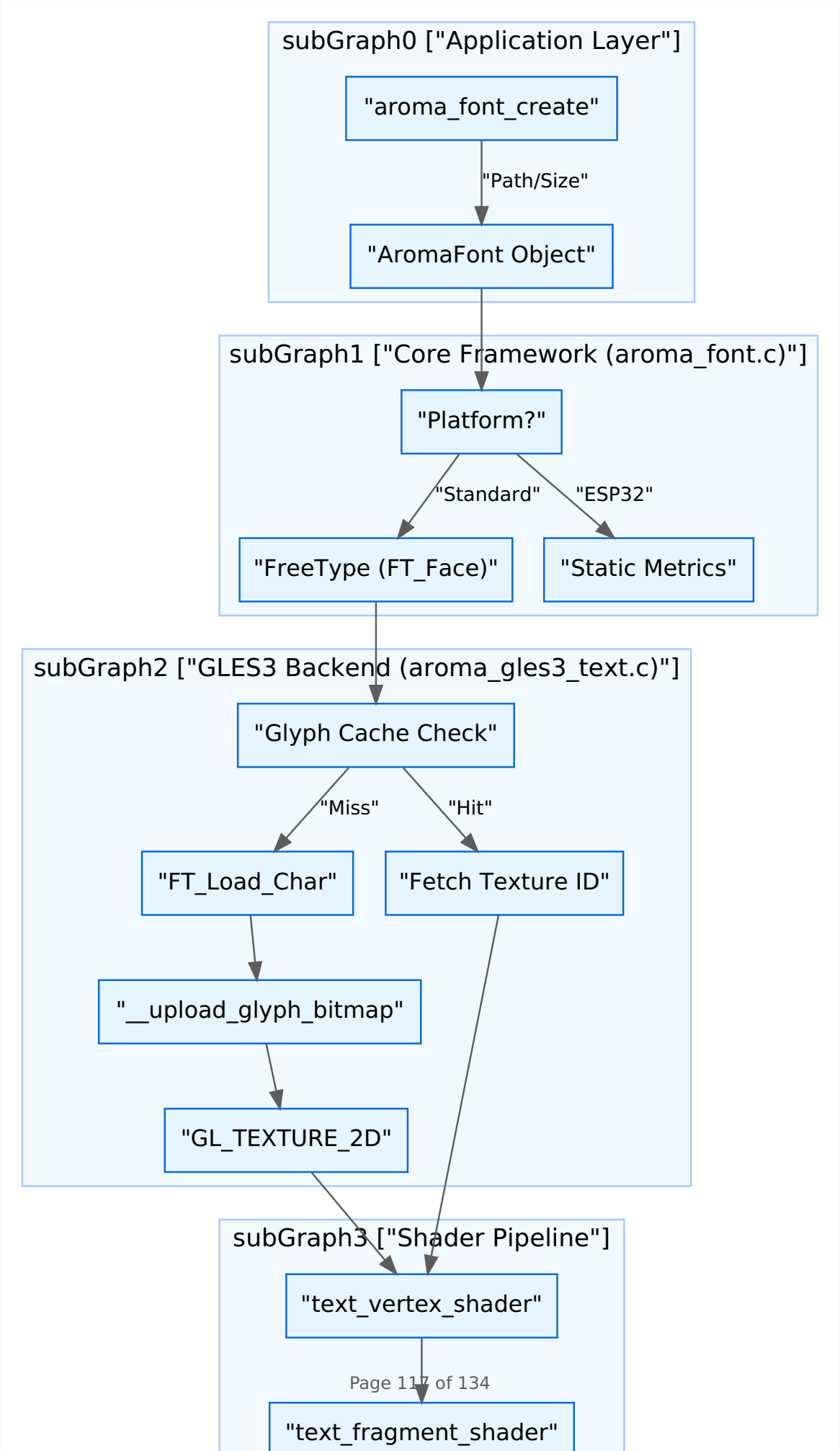
Architecture Overview

The font system is split into a platform-agnostic core and backend-specific implementations. The core manages the `AromaFont` lifecycle and metrics, while the graphics backends handle the actual glyph rasterization and GPU texture caching.

Key Components

- **AromaFont**: An opaque handle representing a loaded font face at a specific pixel size include/aroma_font.h16
- **FreeType Integration**: Uses the FreeType library (via `vendors/freetype`) to parse TTF/OTF files and generate glyph bitmaps src/core/aroma_font.c107-108
- **Embedded Fonts**: Includes a bundled Ubuntu TTF as a byte array for out-of-the-box usage without filesystem dependencies include/aroma_ubuntu_font.h8-9
- **GLES3 LRU Cache**: A per-window glyph cache that stores rasterized characters as OpenGL textures to minimize redundant FreeType calls src/backends/graphics/utils/aroma_gles3_text.c127-147

Data Flow: Font to Screen


subGraph0 ["Application Layer"]
"aroma_font_create"
"Path/Size"
"AromaFont Object"
subGraph1 ["Core Framework (aroma_font.c)"]
"Platform?"
"Standard"
"ESP32"
"FreeType (FT_Face)"
"Static Metrics"
subGraph2 ["GLES3 Backend (aroma_gles3_text.c)"]
"Glyph Cache Check"
"Miss"
"Hit"
"FT_Load_Char"
"Fetch Texture ID"
"__upload_glyph_bitmap"
"GL_TEXTURE_2D"
subGraph3 ["Shader Pipeline"]
"text_vertex_shader"
"text_fragment_shader"

Sources:src/core/aroma_font.c148-174src/backends/graphics/utils/aroma_gles3_text.c152-199src/backends/graphics/utils/helpers_gles3.h123-146

Font Lifecycle and Management

Loading Fonts

Fonts can be loaded from the filesystem or directly from memory buffers. Memory loading is preferred for cross-platform portability and embedded assets.

Function	Description	Source
`aroma_font_create`	Loads font from disk; resolves asset paths automatically.	src/core/aroma_font.c148-174
`aroma_font_create_from_memory`	Loads font from a raw `unsigned char` buffer.	src/core/aroma_font.c176-209
`aroma_font_destroy`	Releases `FT_Face` and frees font memory.	src/core/aroma_font.c217-221

Metrics and Measurement

AromaUI provides APIs to query font metrics essential for layout calculations. All measurements are returned in pixels.

- **Line Height**: The recommended vertical distance between baselines src/core/aroma_font.c239-243

- **Ascender/Descender**: The distance from the baseline to the highest/lowest point of the font src/core/aroma_font.c245-255
- **Text Width**: Calculated by summing the `advance` property of each glyph in a string src/core/aroma_font.c223-236

Sources:include/aroma_font.h41-77

GLES3 Backend Implementation

The GLES3 backend utilizes a specialized `GLES3TextRenderer` to handle high-performance text rendering.

Glyph Caching

To avoid the overhead of re-rasterizing characters every frame, the backend maintains a `GLES3Glyph` cache.

1. **Initial Load**: Characters 32-127 (Standard ASCII) are pre-loaded into textures during `gles3_text_renderer_load_font`src/backends/graphics/utils/aroma_gles3_text.c127-147
2. **Dynamic Loading**: If a codepoint (e.g., UTF-8 or Extended Latin) is missing, `__get_glyph` triggers a FreeType load and uploads a new texture to the GPU src/backends/graphics/utils/aroma_gles3_text.c152-199
3. **Texture Storage**: Glyph bitmaps are stored using `GL_RED` (or `GL_LUMINANCE` on WebGL/Emscripten) to save memory src/backends/graphics/utils/aroma_gles3_text.c95-101

Shaders

Text is rendered as a series of quads. The fragment shader samples the single-channel glyph texture and applies a uniform `textColor`.

⬡ classDiagram

```
classDiagram
    class GLES3TextRenderer {
        +GLuint vao
        +GLuint vbo
        +FT_Face face
        +GLES3Glyph glyphs[MAX_GLYPHS]
        +int glyph_count
    }
    class GLES3Glyph {
        +uint32_t codepoint
        +uint32_t texture_id
        +int width
        +int height
        +int bearing_x
        +int bearing_y
        +int advance
    }
    GLES3TextRenderer *-- GLES3Glyph : caches
```

Sources:src/backends/graphics/utils/aroma_gles3_text.hsrc/backends/graphics/utils/helpers_gles3.h137-146

DP/SP Scaling

AromaUI supports density-independent pixels (**DP**) for layouts and scale-independent pixels (**SP**) for fonts. This ensures that text remains legible across different screen densities.

On Android, the system bridges these calls to `aroma_android_sp_to_px` buzzer/src/main.c28-34 Developers typically define font sizes using SP constants to maintain accessibility standards.

```
#define FONT_TITLE_SP      36
#define FONT_LARGE_SP      28

// Usage in application
int size = sp(FONT_TITLE_SP);
AromaFont* font = aroma_font_create_from_memory(aroma_ubuntu_ttf, aroma_ubuntu_ttf_len, siz
```

Sources:buzzer/src/main.c59-62src/core/aroma_ubuntu_font.c5-104

Platform Specifics: ESP32

On **ESP32** (TFT_eSPI backend), the font system is simplified to reduce memory footprint. It bypasses FreeType and uses fixed metrics for built-in fonts like `FreeSans12pt7b` or `GLCD`src/core/aroma_font.c14-32

- **aroma_font_create**: Instead of loading a file, it matches the `font_path` string to internal identifiers src/core/aroma_font.c34-66
- **aroma_font_get_face**: Returns `NULL` as there is no underlying `FT_Face` object src/core/aroma_font.c99-103

Sources:src/core/aroma_font.c6-103

Testing & Quality

Testing and Quality

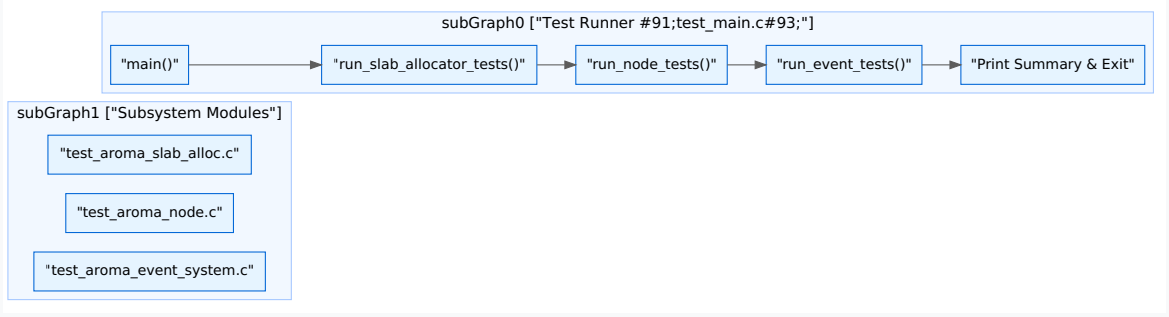
The AromaUI test suite provides a comprehensive set of unit and integration tests designed to ensure the stability of core UI primitives, memory management, and event propagation. The testing architecture is decoupled from any specific platform backend, allowing tests to run in a headless environment (typically Linux) to validate logic without requiring a GPU or display.

Test Suite Architecture

The test suite is organized into modular components targeting specific subsystems. The entry point is `test_main.c`, which orchestrates the execution of different test modules and aggregates results.

Test Execution Flow

Title: AromaUI Test Execution Pipeline



Sources: [tests/test_main.c27-55tests/test_aroma_slab_alloc.c232-242]

Memory Allocator Tests (`test_aroma_slab_alloc.c`)

These tests validate the `AromaSlabAllocator`, which is critical for deterministic memory performance on embedded targets. The tests focus on the multi-cache slab system used for both internal `AromaNode` structures and variable-sized widget data.

Key functions tested:

- **Initialization**: `test_memory_system_init` ensures `aroma_memory_system_init` and `aroma_memory_system_destroy` correctly manage the global allocator state tests/test_aroma_slab_alloc.c37-45
- **Widget Allocation**: `test_widget_allocation` verifies that `aroma_widget_alloc` provides memory for different sizes and that `aroma_widget_free` allows for subsequent re-allocation tests/test_aroma_slab_alloc.c47-74
- **Stress Testing**: `test_widget_allocation_stress` performs 100 interleaved allocations and deallocations of varying sizes to detect fragmentation or pool exhaustion tests/test_aroma_slab_alloc.c76-110
- **Node Pooling**: `test_node_allocation` specifically targets the fixed-size pool used for `AromaNode` structs via `__slab_pool_alloc` tests/test_aroma_slab_alloc.c112-140

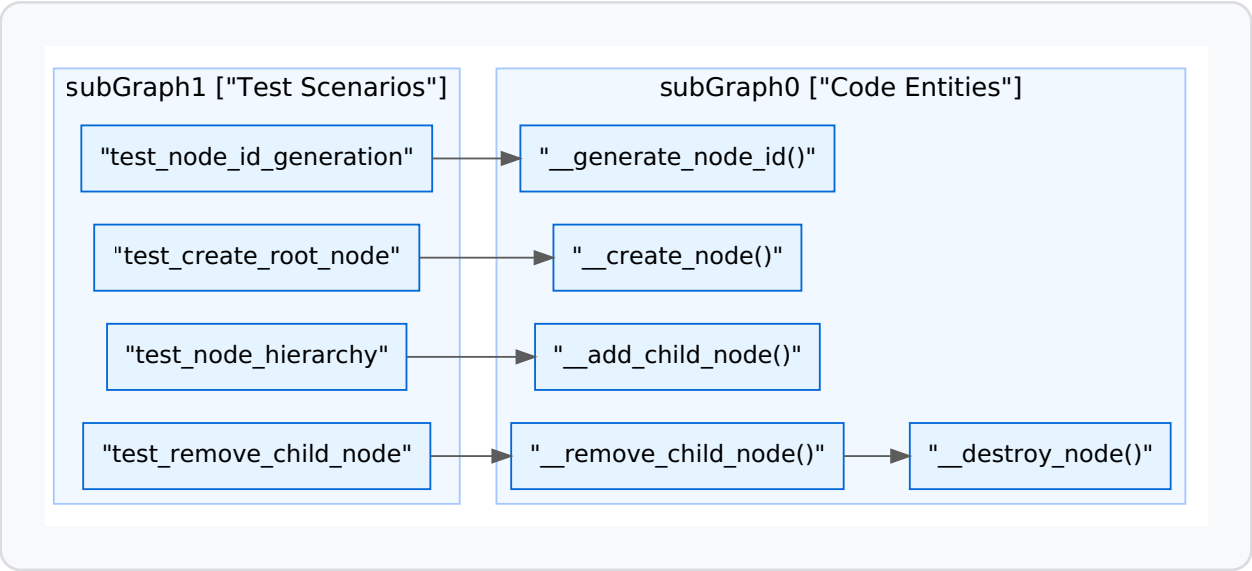
Sources: tests/test_aroma_slab_alloc.c1-242

Scene Graph Tests (`test_aroma_node.c`)

The node system tests verify the integrity of the UI tree, including parent-child relationship management, ID generation, and the cleanup of complex hierarchies.

Node Management Logic

Title: Node Lifecycle and Hierarchy Validation



Key validation logic:

- **ID Uniqueness**: `test_node_id_generation` asserts that every call to `__generate_node_id` returns a strictly increasing, unique identifier tests/test_aroma_node.c59-73
- **Hierarchy Integrity**: `test_node_hierarchy` builds a tree (Root -> Container -> Button) and asserts that `child_count` and `parent_node` pointers are correctly maintained across levels tests/test_aroma_node.c140-160
- **Partial Tree Removal**: `test_remove_child_node` verifies that removing a node from the middle of a child array correctly shifts the remaining siblings to maintain array density tests/test_aroma_node.c162-196

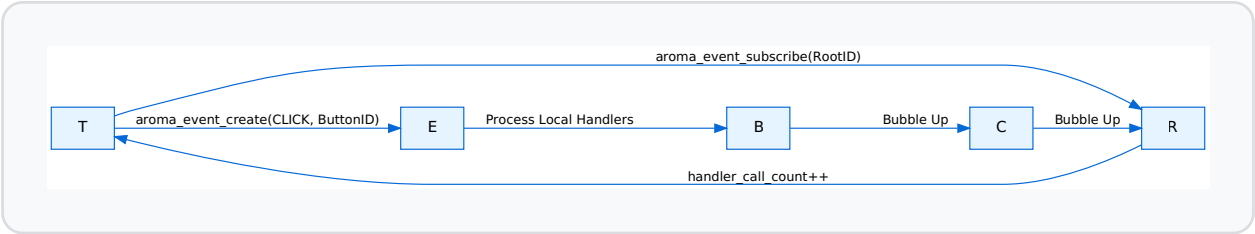
Sources: tests/test_aroma_node.c59-196

Event System Tests (`test_aroma_event_system.c`)

These tests simulate user interaction to verify hit-testing, subscription priorities, and the event bubbling mechanism.

Event Propagation Model

Title: Event Dispatch and Bubbling Test Flow



Key behaviors verified:

- **Subscription**: `test_event_subscription_and_dispatch` ensures that `aroma_event_subscribe` correctly registers a callback and that `aroma_event_dispatch` triggers it when the target ID matches tests/test_aroma_event_system.c95-123
- **Bubbling**: `test_event_bubbling` verifies that an event targeting a leaf node (Button) propagates up to the Root if not consumed tests/test_aroma_event_system.c125-159
- **Consumption**: `test_event_consumption_stops_bubbling` validates that calling `aroma_event_consume` inside a high-priority handler prevents lower-priority or parent-node handlers from receiving the event tests/test_aroma_event_system.c161-199
- **Queueing**: `test_event_queue_processing` tests the asynchronous event path where events are added via `aroma_event_queue` and processed later during `aroma_event_process_queue` tests/test_aroma_event_system.c201-231

Sources: tests/test_aroma_event_system.c95-231

Build Configuration

The test suite is integrated into the CMake build system, allowing for automated testing and memory sanitization.

CMake Integration (`tests/CMakeLists.txt`)

The `aroma_tests` executable links against the core `aroma` library and includes the project headers.

Feature	Implementation
Target Name	`aroma_tests` tests/CMakeLists.txt1
Address Sanitizer	Enabled via `ENABLE_ASAN`, adding `-fsanitize=address` to catch memory leaks and buffer overflows tests/CMakeLists.txt16-22
Test Registration	Uses `add_test` to integrate with CTest tests/CMakeLists.txt24
Includes	Includes both `include/` and the local `tests/` directory for private test headers tests/CMakeLists.txt11-14

Sources: tests/CMakeLists.txt1-24

Glossary

Glossary

This page provides definitions for codebase-specific terms, abbreviations, and domain concepts used throughout the AromaUI framework. It serves as a technical reference for onboarding engineers to understand the internal nomenclature and implementation pointers.

Core Concepts

AromaNode

The fundamental building block of the AromaUI scene graph. Every UI element (windows, buttons, containers) is an `AromaNode`. It manages the hierarchical parent-child relationship (limited to 64 children for performance) and maintains spatial properties like `AromaRect`.

- **Implementation:**include/aroma_node.h1-50
- **Lifecycle:** Initialized via `__node_system_init`src/core/aroma_ui_impl.c165

AromaBackendABI

The "Application Binary Interface" proxy layer that decouples the core framework from specific graphics and platform implementations. It routes generic draw calls (e.g., `draw_rect`) to the active backend (GLES3, Vulkan, or TFT_eSPI).

- **Implementation:**src/backends/aroma_abi.c1-100
- **Access:** Accessed via the global `aroma_backend_abi` struct src/core/aroma_ui_impl.c33

DrawList

A command-buffer system used for deferred rendering. Instead of immediate execution, UI nodes record drawing commands into an `AromaDrawList`. This allows for Z-index sorting, frustum culling, and batching before the final flush to the GPU.

- **Implementation:**src/core/aroma_drawlist.c1-150
- **Usage:** Collected during the frame update via `collect_draw_tasks`src/core/aroma_ui_impl.c71-73

Dirty-Region Tracking

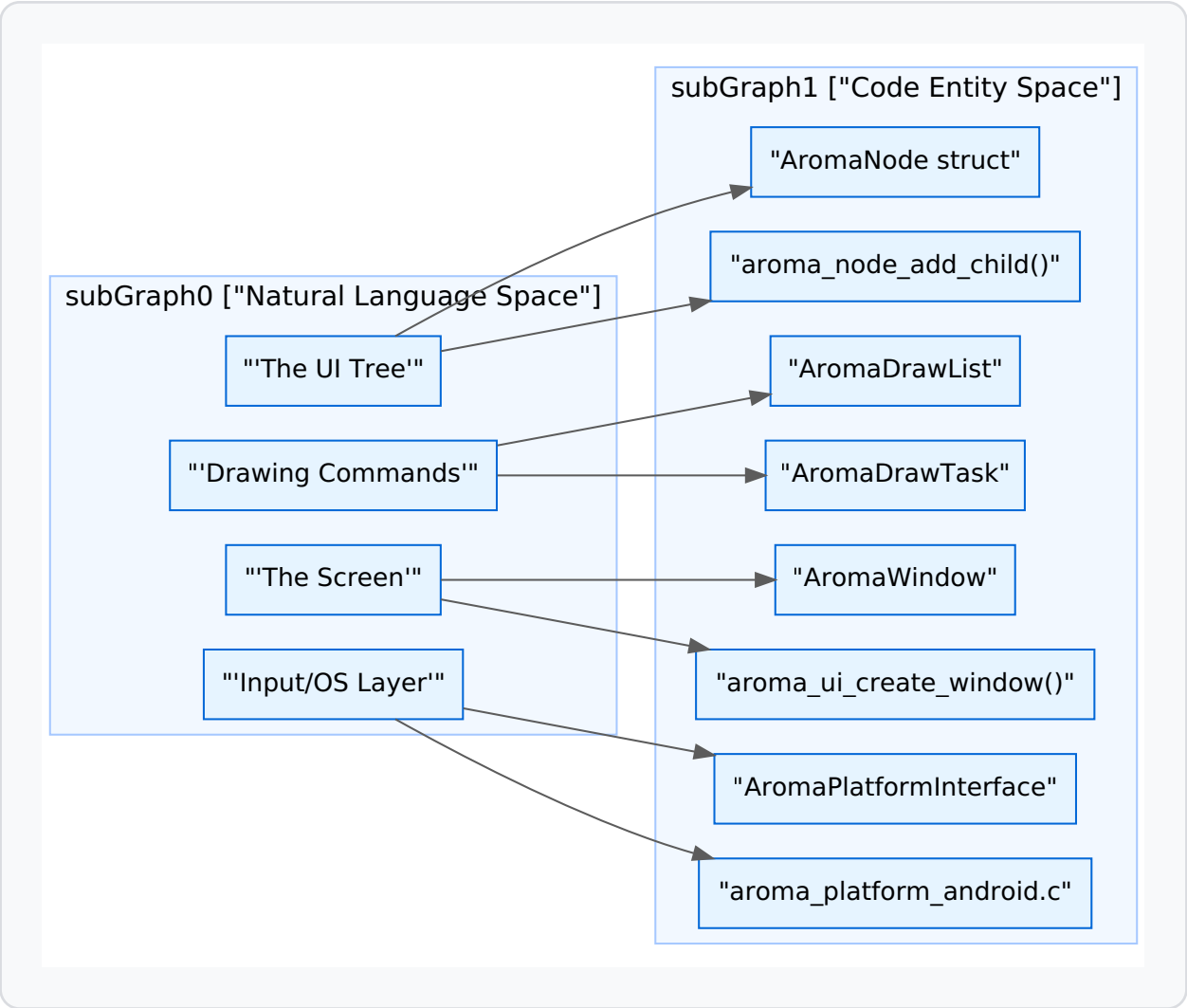
An optimization technique where only portions of the screen that have changed are re-rendered. Nodes are marked "dirty" via `aroma_node_invalidate`, adding them to a global dirty list.

- **Implementation:**src/core/aroma_node.c120-150
- **Trigger:** `aroma_node_invalidate(g_main_window)`src/core/aroma_ui_impl.c221

System Architecture Mapping

The following diagram maps high-level system concepts to their specific code entities and file locations.

Natural Language to Code Entity Space



Sources:include/aroma_node.h1-20src/core/aroma_ui_impl.c54-65src/backends/platforms/aroma_platform_android.c1-50include/aroma_drawlist.h1-30

Technical Terms & Abbreviations

Term	Definition	Code Pointer
ABI	Aroma Backend Interface; the abstraction layer for graphics/platform.	`aroma_backend_abi` src/backends/aroma_abi.c10-15
Slab Allocator	A memory management system for fixed-size node allocations, critical for embedded targets.	src/core/aroma_slab_alloc.c1-100
Hit-Testing	The process of determining which node an input event (touch/click) belongs to.	`aroma_event_hit_test` src/core/aroma_event.c50-80
Z-Layer	Integer priority determining the stack order of UI elements.	`aroma_node_set_z_index` examples/car_infotainment/vehicle_view.c67
DP/SP	Density-independent Pixels / Scale-independent Pixels (for fonts).	`android_sp_to_px` src/backends/platforms/aroma_platform_android.c98-99
Vosk	The offline speech recognition engine used for voice commands.	`vosk_model_new` examples/car_infotainment/voice_control.c153
Telemetery Bridge	Shared memory interface for IPC (Inter-Process Communication) of vehicle data.	`telemetry_bridge_open` examples/car_infotainment/main.c70

Domain Concepts: Android Integration

AromaUI treats Android as a specific platform backend that utilizes JNI (Java Native Interface) to access system services.

AromaHelper

The Java-side singleton that handles Android-specific tasks like Bluetooth scanning, Toast messages, and Preferences.

- **Template:**tools/cli/templates/android/app/src/main/java/AromaHelper.java.tpl1-200
- **JNI Cache:** Native code caches method IDs in `AromaHelperCache` for high-speed calls. src/backends/platforms/aroma_platform_android.c133-171

Native App Glue

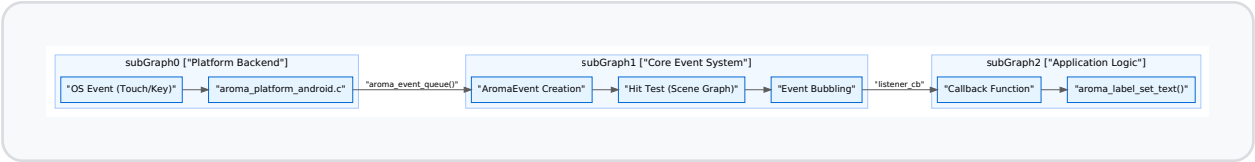
The standard Android NDK utility used to manage the `android_app` state and the `ANativeWindow` lifecycle.

- **Pointer:** `g_app` src/backends/platforms/aroma_platform_android.c50
- **Callback:** `aroma_android_set_app` src/core/aroma_ui_impl.c31-39

Data Flow: Event to Action

This diagram illustrates how a physical interaction travels through the system to trigger application logic.

Event Pipeline Data Flow



Sources:src/core/aroma_event.c1-100src/backends/platforms/aroma_platform_android.c86-114examples/car_infotainment/vehicle_view.c6-14

Embedded & Web Terms

- **TFT_eSPI:** A specific backend for ESP32/embedded displays that uses tiled rendering to fit within small RAM constraints. src/backends/graphics/aroma_graphics_tft_espi.cpp1-50
- **Emscripten/WASM:** The toolchain and target for running AromaUI in a web browser.
- **DPR:** Device Pixel Ratio handling for high-DPI web screens. src/core/aroma_ui_impl.c85-88
- **Fetch:** `emscripten_fetch` used for async tile loading in the Map widget. src/widgets/aroma_map.c21
- **OSRM:** Open Source Routing Machine; the backend API used by `aroma_map.c` to calculate navigation paths. src/widgets/aroma_map.c240-260

Sources:

- src/core/aroma_ui_impl.c1-250
- src/backends/platforms/aroma_platform_android.c1-230
- include/aroma_node.h1-50
- examples/car_infotainment/main.c1-125
- src/widgets/aroma_map.c1-260